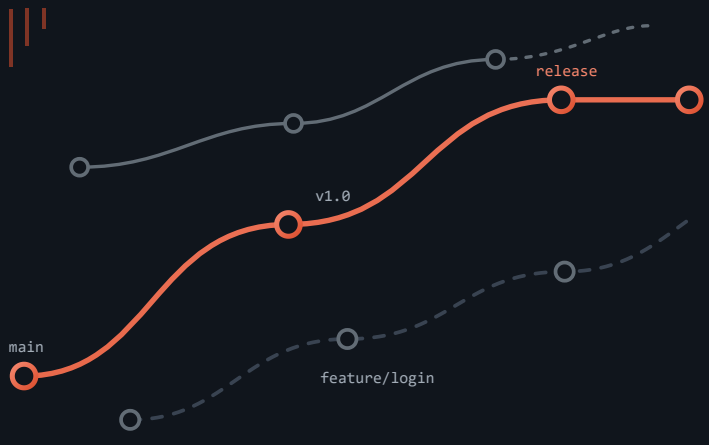
WINDOWS · macOS · LINUX

git.

安装与配置完全指南

从下载安装包到第一次成功 push：逐页拆解安装选项，讲清 PATH、换行符、SSH、凭据与代理，并给出可直接照做的验收与排错清单。

WINDOWS 10 / 11　　macOS 12+　　LINUX



READ ME FIRST / 使用说明

关于本指南

这不是一份“下一步、下一步、完成”的截图合集。安装 Git 真正容易出问题的地方，往往藏在安装向导的后半程：PATH 选错、换行符悄悄改写文件、SSH 与 HTTPS 混用、代理只配了一半。本指南把这些决定逐项讲清楚，并给出安装后的验收方法。

DOCUMENT	BASELINE	UPDATED
全平台安装与配置手册	**Git 2.56.0** 2026-10-26 发布	**2026 年 10 月** 版本号以官网为准

适合谁读

- 第一次安装 Git，希望一次配置到位的学生与开发者。
- 需要在 Windows、macOS、Linux 之间切换，想统一使用习惯的人。
- 要为公司或实验室写安装说明、做新机器初始化的维护者。
- 已经装过 Git，但被 PATH、SSH、换行符或代理反复困扰的人。

本指南覆盖什么

- 三大平台的安装路线、下载渠道与安装向导逐项解释。
- 身份、默认分支、编辑器、换行符、凭据、代理等首轮配置。
- SSH 密钥、多账号、HTTPS 令牌与远程仓库接入。
- 十项验收、诊断命令、常见报错与干净卸载。

怎么读最省时间

ROUTE 01

第一次安装

读第 1 章建立概念，然后按系统跳到第 2、3 或 4 章；装完直接做第 8 章验收。

ROUTE 02

已经装好，想少踩坑

从第 5 章开始配置身份与换行符，再用第 6 章接通 GitHub / GitLab / Gitee。

ROUTE 03

现在就有报错

先看第 9 章的报错索引；涉及权限与路径的，配合第 8 章诊断命令定位。

ROUTE 04

团队 / 教学使用

附录 A 可直接打印成命令卡，附录 B 是可复制的配置模板，附录 C 是官方链接。

01 版本与路径唯一
知道自己在用哪一个 git.exe

02 身份与换行符正确
提交可归属，文件不被改写

03 远程认证已打通
SSH 或 HTTPS 至少一条可用

命令与排版约定

写法	含义
`$ git status`	`$` 是提示符，不要输入；在 Git Bash、Terminal 或 PowerShell 中输入后面的命令。
`<YOUR_NAME>`	尖括号表示占位符，替换为你的内容，不要保留尖括号。
`# 注释`	代码块中的注释，不需要输入；Windows 路径使用反斜杠，Git Bash 中也可写成 `/c/...`。
管理员	Windows 指“以管理员身份运行”；macOS / Linux 指命令前加 `sudo`。

> **开始之前：三条安全边界**
>
> **不要**为了“让报错消失”而全局关闭 TLS 校验（`http.sslVerify=false`）或随手执行 `git reset --hard`、`git push --force`。前者会让凭据与代码暴露在中间人攻击下，后者可能覆盖他人提交。
>
> **不要**把私钥、访问令牌、`.env` 或包含密码的远程 URL 写进仓库。本指南会在第 6 章与第 10 章说明更安全的替代做法。

五个最常见的安装误区

常见说法 / 做法	实际后果	在哪里解决
“Git 就是 GitHub 客户端。”	断网时不敢提交；把远程仓库当成唯一备份。	1.1 节、7.1 节
安装向导一路点 Next。	PATH 指向旧版本；换行符悄悄改写脚本；默认编辑器不会用。	2.2 节、5.2 节
继续用账号密码推 GitHub。	提示 403 或“Password authentication was removed”。	6.4 节、6.5 节
电脑里装了三套 Git，却不知道在用哪套。	版本、配置、SSH 行为不一致；升级后“时好时坏”。	8.2 节、9.1 节
只验证 `git --version` 就算装完。	能显示版本，却不能提交、不能认证、不能推送。	第 8 章十项验收

表 0-1 安装阶段的高频误区。它们大多不是“不会命令”，而是配置决策没有被验证。

版本说明：本指南以 Git 2.56.0（2026 年 10 月 26 日发布）为基线，适用于 Git 2.40 及以上版本。Git for Windows 安装向导的页面标题与选项顺序可能随小版本微调，请以“选项关键词”而不是固定页码对照。所有命令均在官方文档语义下给出；不同发行版的包名与默认配置可能不同，正文会单独标注。

目录

CONTENTS / 13 CHAPTERS & APPENDICES

PART I 准备与安装

01 安装之前：先把 Git 放到位 6
1.1 Git、GitHub 与“三个区域” 6
1.2 版本、安装包与获取渠道 6
1.3 安装前的系统检查 7
1.4 选择适合你的安装路线 8

02 Windows：Git for Windows 逐页安装 9
2.1 下载与校验 9
2.2 安装向导的关键选择 10
2.3 首次启动与路径检查 14
2.4 升级、静默安装与卸载 15

03 macOS：Xcode CLT、官方包与 Homebrew 17
3.1 先检查系统自带的 Git 17
3.2 三条安装路线 18
3.3 Shell、PATH 与 Gatekeeper 20
3.4 升级与卸载 21

04 Linux：包管理器与源码编译 22
4.1 发行版命令速查 22
4.2 Ubuntu 官方 PPA 与源码编译 23
4.3 验证、升级与卸载 24
4.4 容器、WSL 与最小化环境 25

PART II 配置与接入

05 安装后的第一轮配置 27
5.1 配置层级与身份信息 27
5.2 默认分支、编辑器与换行符 28
5.3 凭据、代理与常用偏好 30
5.4 配置文件备份与迁移 32
5.5 配置验收：四条命令 33

06 SSH 与 HTTPS：安全连接远程仓库 34
6.1 选 SSH 还是 HTTPS 34
6.2 生成与保管 SSH 密钥 34
6.3 ssh-agent 与多账号配置 35
6.4 添加公钥并测试连接 37
6.5 HTTPS 令牌与常见认证错误 38

PART III 实战、验收与维护

07 第一个仓库：从 init 到 push 40
7.1 创建仓库并完成第一次提交 40
7.2 连接远程仓库与克隆 41
7.3 .gitignore 与分支协作 43
7.4 撤销、回退与冲突处理 44
08 验收：确认安装真的可用 46
8.1 十项验收清单 46
8.2 诊断命令与正常输出 47
8.3 网络与代理连通性测试 48
09 常见问题：从报错到修复 50
9.1 安装与 PATH 类问题 50
9.2 提交、推送与换行符问题 51
9.3 SSH、HTTPS 与网络问题 51
9.4 权限、长路径与其他问题 52
9.5 排错记录模板 53
10 维护与安全 54
10.1 升级策略与版本管理 54
10.2 配置、密钥与令牌安全 55
10.3 干净卸载与迁移 56

APPENDICES

A 命令速查表 57
B 推荐配置模板 59
C 官方资源与术语表 61

页码为条目所在页；点击条目可跳转。目录在排版完成后自动生成。

FOUNDATION / 安装之前

01

安装之前：先把 Git 放到位

Git 不只是一个“下载后点下一步”的工具。它把项目的完整历史放在本机，通过暂存区决定每次提交的内容，再与远程仓库同步。理解这条链路，后面的 PATH、SSH、换行符选项才不会变成死记硬背。

预计 12 分钟 | 无需安装 | 概念与路线选择

1.1 Git、GitHub 与“三个区域”

Git 是安装在你电脑上的分布式版本控制系统；**GitHub、GitLab、Gitee** 是托管 Git 仓库的网站。没有网络，Git 仍可在本地提交、查看历史、切换分支；只有 `clone`、`fetch`、`push` 这类与远程同步的操作才需要联网。



日常操作可以概括为：修改工作区文件 → 用 `git add` 放入暂存区 → 用 `git commit` 写入本地仓库 → 用 `git push` 同步到远程。任何一步都可以单独撤销，但撤销方式不同；第 7 章会把这条链路完整跑一遍。

LOCAL

本地仓库

项目根目录下的隐藏文件夹 `.git`，保存对象、引用、配置与日志。删掉它，项目文件还在，但版本历史全部消失。

REMOTE

远程仓库

团队共享的副本，默认习惯命名为 `origin`。它只是“另一个仓库”，不是备份服务的同义词；重要项目仍应做独立备份。

> **一个常见误解**
>
> “把代码上传到 GitHub”并不等于“Git 安装成功”。安装是否成功，取决于本机的 `git` 命令能否运行、配置是否写入正确的层级、网络与认证是否打通。第 8 章会用十项检查分别验证这三层。

1.2 版本、安装包与获取渠道

截至 2026 年 10 月，Git 官方最新稳定版为 **2.56.0**，Git for Windows 对应安装器为 **2.56.0(4)**。不必追求“刚发布的第一个版本”；生产环境更重要的是可追溯、可回滚。下表是常见获取渠道的取舍。

渠道	适合场景	优点	注意事项
官网安装包	个人电脑、第一次安装	版本新、选项完整、带卸载器	只从 git-scm.com 或 Git for Windows 官方发布页下载；核对架构与文件签名。
系统包管理器	Linux、macOS、企业批量部署	可脚本化、随系统统一升级	版本可能落后；第三方仓库可能改动编译选项，需确认来源可信。
winget / Chocolatey	Windows 快速装机	一条命令完成，便于自动化	包由社区维护，可能滞后于官方安装器；升级前先看版本号。
便携版	无管理员权限、U 盘演示、隔离环境	解压即用，不写系统目录	需要手动加入 PATH；配置与凭据仍可能写入用户目录。
源码编译	需要特定版本、特定编译选项	可控性最高	需装编译依赖；不会自动更新；卸载要回到源码目录执行。

表 1-1 安装渠道对比。普通用户优先“官网安装包”；Linux 服务器优先发行版仓库。

版本号怎么读

Git 使用 `主版本.次版本.修订号`。例如 `2.56.0`：次版本增加通常带来新功能，修订号只修缺陷。Git for Windows 的 `2.56.0(4)` 表示基于 Git 2.56.0 的第 4 个 Windows 构建，括号里的数字不是 Git 本体版本。看到 `2.56.0.windows.4` 也是同一意思。

> **选择原则**
>
> **个人电脑**：官网最新稳定版。**公司电脑**：跟随公司软件源或已批准版本，不要自行混装。**服务器**：发行版仓库 + 定期安全更新。**需要复现实验**：固定次版本号，并在项目文档中记录。

1.3 安装前的系统检查

安装失败很少是因为 Git 本身，更多是权限、磁盘、旧版本残留或安全软件拦截。花两分钟完成下面五项检查，能省掉后面很多来回。

- [ ] **系统版本。**Windows 建议 10 22H2 / 11；macOS 建议 12 及以上；Linux 使用仍受支持的发行版。
- [ ] **管理员权限。**系统级安装需要；便携版不需要，但可能无法写入受保护目录。
- [ ] **旧版本与 PATH。**先运行 `git --version`、`where git`（Windows）或 `which -a git`，记录现有版本。
- [ ] **安全软件。**若安装器被静默拦截，先看隔离记录，不要把整个目录永久排除。
- [ ] **处理器架构。**确认是 x64 还是 ARM64；下载错架构的安装包会直接拒绝运行。
- [ ] **磁盘空间。**预留至少 500 MB；源码编译建议 2 GB 以上。
- [ ] **网络与代理。**能打开 git-scm.com；公司网络需知道代理地址与端口。
- [ ] **备份。**已有 `~/.gitconfig`、SSH 密钥或未提交改动时，先备份再升级。

一分钟记录现状

在安装前把结果贴到备忘里，升级出问题时可以快速对比：

```
# Windows PowerShell
PS> git --version
PS> where.exe git
PS> $env:PROCESSOR_ARCHITECTURE

# macOS / Linux
$ git --version
$ which -a git
$ uname -m
```

> **旧版本不一定需要先卸载**
>
> Windows 上升级 Git for Windows 通常可以直接覆盖安装，安装器会保留你的全局配置；macOS 的 Homebrew 版本与系统自带版本可以并存；Linux 包管理器会原地升级。只有在“多个来源混装、PATH 指向混乱”时，才需要先清理旧版本。不要在未确认路径的情况下直接删除 `/usr/bin/git` 或系统目录。

1.4 选择适合你的安装路线

后面的三章分别对应三个平台。如果你只使用一个系统，直接跳到对应章节；如果同时使用多个系统，建议先读 Windows 一节，因为它的安装选项最多，也最容易把跨平台习惯带偏。

你的情况	推荐路线	理由与下一步
Windows 个人电脑	Git for Windows 官方安装器	选项完整、带 Git Bash 与凭据管理器；按第 2 章逐页选择。
Windows 公司电脑	公司软件中心 / winget	先确认已批准版本；安装后核对 `git --version` 与 PATH，别与旧版并存。
macOS 只做少量提交	Xcode Command Line Tools	系统自带、零额外维护；版本可能较旧，按第 3.1 节检查。
macOS 长期开发	Homebrew 安装 Git	版本新、升级方便；注意 `/opt/homebrew/bin` 在 PATH 中的顺序。
Ubuntu / Debian 服务器	`apt install git`	稳定、可审计；需要新版本再用官方 PPA 或源码。
Docker / CI 镜像	基础镜像内安装 git + openssh-client	注意镜像精简后缺少 CA 证书与 SSH 客户端，按第 4.4 节补齐。

表 1-2 按使用场景选择安装路线。选定后直接进入对应章节，不必通读全部平台。

NEXT / WINDOWS

下一章会从下载页开始，逐页解释 Git for Windows 安装向导中的 12 个关键选择，并给出“推荐选项 + 适用条件 + 选错后的表现”。如果你不使用 Windows，可以跳到第 3 章或第 4 章。

WINDOWS / GIT FOR WINDOWS

02

Windows：Git for Windows 逐页安装

Git for Windows 把 Git、Git Bash、MinTTY、OpenSSH 与 Git Credential Manager 打包成一个安装器。真正需要你判断的只有几页：PATH、SSH、HTTPS 后端、换行符和凭据助手。这一章逐项拆开，并给出推荐组合。

预计 25 分钟 | 需要管理员权限 | Windows 10 / 11 · x64 / ARM64

Git for Windows 的安装器基于 Inno Setup，界面朴素，但每个选项都会改变后续行为。不同小版本的页面标题可能略有差异；下面用“页面关键词 + 选项原文”对照，不依赖固定第几屏。第一次安装建议完整读一遍 2.2 节，之后升级可以只看 2.4 节。

2.1 下载与校验

只从两个地方下载：Git 官网的 Windows 页面，或 Git for Windows 的 GitHub Releases。搜索引擎广告位、网盘转载和“绿色版”都不要用。官网会自动识别系统，但下载前仍要确认架构。

1. **确认系统架构**

   在 PowerShell 中运行 `$env:PROCESSOR_ARCHITECTURE`。常见结果：`AMD64` 对应 x64 安装包；`ARM64` 对应 ARM64 安装包。不要因为“ARM64 也能跑 x64”就随便选，原生包更稳定。

2. **下载官方安装器**

   打开 `git-scm.com/download/win`，选择 **64-bit Git for Windows Setup**；ARM 设备选择 **ARM64 Setup**。文件名类似 `Git-2.56.0.4-64-bit.exe`，括号中的构建号可能更新。

3. **核对文件来源与完整性**

   右键安装器 → “属性” → “数字签名”，确认签名有效且签名者为 Git for Windows 的发布者。若发布页提供 SHA-256，可用 `Get-FileHash .\Git-*.exe -Algorithm SHA256` 对照；不一致就重新下载。

4. **关闭占用 Git 的程序**

   升级时先关闭 VS Code、Windows Terminal、Git Bash、IDE 和正在运行的构建任务。文件被占用会导致安装器无法替换 `git.exe`，最常见的结果是“装完了但版本没变”。

RECOMMENDED

标准安装器

带开始菜单、右键菜单、卸载器与 Git Bash，适合绝大多数个人电脑。安装到 `C:\Program Files\Git`，需要管理员权限。

NO ADMIN

便携版

解压到普通目录即可运行，适合无管理员权限的实验室电脑。需要手动把 `...\cmd` 加入 PATH，且不会自动创建右键菜单。

也可以使用 winget

Windows 10 1809 及以上可通过 App Installer 使用 `winget`；若系统没有，先从 Microsoft Store 安装“应用安装程序”。它能一条命令完成安装与升级，适合批量装机；但包由社区维护，版本可能比官网滞后几天。安装后仍要用 `git --version` 核对。

```
PS> winget install --id Git.Git -e --source winget
# 已安装时升级
PS> winget upgrade --id Git.Git -e --source winget
# 查看包信息与可用版本
PS> winget show --id Git.Git -e
```

> **不要从三个渠道同时安装**
>
> 官网安装器、winget、Chocolatey 与便携版可能各自写一套 PATH 和卸载信息。混装后 `where.exe git` 可能列出多条路径，实际运行的却是旧版本。选定一种渠道，卸载其余版本，再继续。

2.2 安装向导的关键选择

双击安装器后，前几屏是许可协议、安装路径和组件选择；后半程才是重点。下面按页面顺序说明。如果你只想快速装好，先看表 2-1 的推荐组合，再读标有“关键”的几节。

向导页面（关键词）	推荐选择	一句话理由
License Agreement	I accept the agreement	GPLv2；不接受无法继续。
Select Destination Location	`C:\Program Files\Git`	避免中文、空格和 OneDrive 同步目录。
Select Components	Git Bash Here + Git LFS（按需）	右键菜单提高效率；LFS 只在大文件项目需要。
Select Start Menu Folder	默认 `Git`	保留即可，便于找到 Git Bash。
Default Editor	VS Code（已安装）或 Nano	必须能阻塞等待提交信息；不要选“无”。
Initial Branch Name	Override: `main`	与 GitHub / GitLab 新仓库默认一致。
PATH Environment	Git from the command line and also from 3rd-party software	让 PowerShell、VS Code、IDE 都能调用 `git`。
SSH Executable	Use bundled OpenSSH	与 Git for Windows 配套，少一层变量。
HTTPS Transport Backend	Use the native Windows Secure Channel library	直接使用 Windows 证书库，企业网络更省心。
Line Ending Conversions	Windows 单平台：Windows-style；跨平台：as-is + input	决定文件在检出和提交时是否转换 CRLF。

向导页面（关键词）	推荐选择	一句话理由
Terminal Emulator	Use MinTTY	Unicode、颜色、窗口缩放都更好。
Default `git pull` Behavior	Default (fast-forward or merge)	对新手最不容易丢提交。
Credential Helper	Git Credential Manager	HTTPS 推送不用反复输入令牌。
Extra / Experimental Options	文件系统缓存开；符号链接按需；FSMonitor 可不开	缓存提升性能；符号链接会引入跨平台差异。

表 2-1 Git for Windows 安装向导推荐组合。实际页面可能合并或拆分，按关键词对应即可。

2.2.1 路径、组件与开始菜单

安装路径保持默认 `C:\Program Files\Git`。不要装到中文用户名目录、OneDrive 同步目录或带空格的路径下：不是 Git 本身不支持，而是某些脚本、IDE 插件和旧工具在拼接路径时容易出错。若必须自定义，使用纯英文、无空格、非同步盘路径。

组件页按需勾选：

- **Windows Explorer integration**：勾选 **Git Bash Here** 与 **Git GUI Here**。Windows 11 的右键菜单可能收在“显示更多选项”里，这是系统行为。
- **Git LFS (Large File Support)**：仓库里有 PSD、视频、数据集等大文件时再勾。它会把大文件替换为指针，需要远程仓库支持 LFS。
- **Associate .git* configuration files**：让 `.gitignore`、`.gitattributes` 用默认编辑器打开，建议勾选。
- **Associate .sh files to be run with Git Bash**：经常双击运行 shell 脚本再勾；否则可能改变 `.sh` 的默认打开方式。
- **Use a TrueType font in all console windows**：勾选可改善 Git Bash 的中文与符号显示。
- **Check daily for Git for Windows updates**：个人电脑可勾；企业电脑建议关闭自动检查，由软件中心统一升级。

2.2.2 默认编辑器与初始分支

默认编辑器决定 `git commit`（不写 `-m` 时）和 `git config --edit` 用哪个程序写文本。推荐 VS Code，但必须带 `--wait`：

```
# 安装器里选择 “Use Visual Studio Code as Git's default editor”
# 若手动配置，--wait 不能漏
$ git config --global core.editor "code --wait"

# 没装 VS Code 时，Nano 是最省事的选择
$ git config --global core.editor nano
```

如果只写 `code` 而不写 `--wait`，VS Code 会在后台启动并立刻把控制权还给 Git，Git 以为你提交了空信息，于是放弃提交。看到“Aborting commit due to empty commit message”时，先检查这一点。

初始分支名建议覆盖为 `main`。Git 本身仍允许 `master`，但 GitHub、GitLab、Gitee 新建仓库普遍使用 `main`；统一后少一次 `git branch -M`。若公司规范明确使用 `master`，跟随公司规范，不要为了“新”而制造不一致。对应配置是 `git config --global init.defaultBranch main`，可用 `git config --global --get init.defaultBranch` 复核。

2.2.3 PATH：最关键的一页

这一页决定“在哪些终端里能直接输入 `git`”。三个选项的差异如下：

选项原文	效果	适用与风险
Use Git from Git Bash only	只有 Git Bash 能用	不影响系统 PATH，最保守；但 PowerShell、VS Code 终端、IDE 都找不到 `git`，不推荐普通开发使用。
Git from the command line and also from 3rd-party software	CMD、PowerShell、IDE 都能用	**推荐。**安装器把 `C:\Program Files\Git\cmd` 加入 PATH，但不覆盖 Windows 自带的 `find`、`sort` 等命令。
Use Git and optional Unix tools from the Command Prompt	额外加入 Unix 工具	会把 `find.exe`、`sort.exe` 等放进 PATH，可能改变现有脚本行为。除非明确需要，不要选。

表 2-2 PATH 三个选项。选错的典型表现是：Git Bash 里正常，PowerShell 里提示“无法将 git 项识别为 cmdlet”。

安装完成后用 `where.exe git` 验证顺序。输出中应优先出现 `C:\Program Files\Git\cmd\git.exe`；如果第一行是旧版本或其他渠道的路径，说明 PATH 还没理顺。

2.2.4 SSH 与 HTTPS 后端

SSH executable 推荐 **Use bundled OpenSSH**。Git for Windows 自带的 `ssh.exe` 与 Git Bash 的路径、配置和权限模型一致；选外部 OpenSSH 通常用于公司已经统一使用 Windows OpenSSH 或 PuTTY/Pageant 的场景。混用两套 agent 的典型症状是：密钥明明加了，`ssh -T` 却仍提示 `Permission denied (publickey)`。

HTTPS transport backend 推荐 **Use the native Windows Secure Channel library**。它直接使用 Windows 证书存储，企业根证书、代理证书和系统更新都能复用；**OpenSSL** 使用安装包自带的 CA 证书，适合需要自定义 CA 文件、或跨平台统一 `http.sslCAInfo` 的场景。若公司网络做了 TLS 拦截，Secure Channel 通常更少报 `SSL certificate problem`。

> **SSH 与 HTTPS 不是二选一装哪个**
>
> 安装器问的是“Git 调用哪个 SSH 程序”和“HTTPS 用哪个 TLS 后端”，不是“你以后只能用 SSH 或 HTTPS”。两种协议可以同时使用：个人仓库走 SSH，公司内网仓库走 HTTPS，互不冲突。第 6 章会讲怎么选、怎么配。

2.2.5 换行符：最容易被忽视的一页

Windows 用 CRLF（`\r\n`），Linux/macOS 用 LF（`\n`）。Git 可以在检出和提交时转换，但转换规则必须与团队一致。安装器提供三个选项：

选项	core.autocrlf	行为与适用场景
Checkout Windows-style, commit Unix-style line endings	`autocrlf=true`	检出时 LF → CRLF，提交时 CRLF → LF。适合 Windows 单平台、或团队成员主要用 Windows 且仓库没有 `.gitattributes` 的情况。
Checkout as-is, commit Unix-style line endings	`autocrlf=input`	检出不改，提交时把 CRLF 转为 LF。适合跨平台协作、脚本/Docker/CI 场景。**跨平台团队优先选这个。**
Checkout as-is, commit as-is	`autocrlf=false`	完全不转换。只有仓库已经用 `.gitattributes` 精确管理换行符，或你清楚后果时才选。

表 2-3 换行符转换。无论选哪个，**仓库内的 `.gitattributes` 优先级更高**，团队项目应把规则写进仓库而不是只靠本机配置。

常见警告 `warning: LF will be replaced by CRLF` 不是错误，它在提示“提交/检出时会发生转换”。如果文件是 `.sh`、`Dockerfile`、`Makefile` 或 CI 配置，错误的 CRLF 会让 Linux 端报 `/bin/bash^M: bad interpreter`。这时应在仓库根目录加 `.gitattributes`：

.gitattributes

```
# 仓库内统一 LF，Windows 脚本例外
* text=auto eol=lf
*.bat text eol=crlf
*.cmd text eol=crlf
*.ps1 text eol=crlf

# 二进制文件不做换行转换
*.png binary
*.jpg binary
*.pdf binary
```

2.2.6 终端、pull 与凭据助手

Terminal Emulator 推荐 MinTTY

MinTTY 对 Unicode、颜色、复制粘贴和窗口缩放的支持更好。若公司有依赖旧控制台的自动化工具，才选 **Windows' default console window**。

Default git pull behavior Default

Default (fast-forward or merge) 在分叉时创建合并提交，不会改写历史，最适合新手。选择 **Rebase** 需要理解冲突与变基；**Only ever fast-forward** 则会在分叉时直接失败。

Credential Helper Git Credential Manager

GCM 会把 HTTPS 凭据存入 Windows 凭据管理器，支持 GitHub/GitLab/Azure DevOps 的 OAuth 与双因素认证。**不要选 None**，否则每次 `fetch` / `push` 都会弹窗或直接失败。只用 SSH 的机器可以选 None，但后续切 HTTPS 时仍要补配。

2.2.7 Extra 与 Experimental 选项

- **Enable file system caching**：建议勾选，能明显改善大仓库的状态检查速度。若遇到文件状态不同步、杀毒软件冲突，再尝试关闭。

- **Enable symbolic links**：只有仓库确实包含符号链接，且你理解 Windows 的开发者模式/管理员权限限制时才勾。普通项目勾了不会让 Git 更好用，反而可能在跨平台检出时产生“链接变成文本文件”的问题。
- **Enable experimental built-in file system monitor**：超大仓库可改善 `git status` 性能；普通项目不需要。开启后若状态异常，先关闭它再排查。
- **Enable experimental support for pseudo consoles**：只在 Git Bash 中运行需要交互式控制台的程序（如某些 Python/Node 调试器）时考虑。普通 Git 使用不需要。

> **一键推荐组合（个人 Windows 电脑）**
> 默认路径 + Git Bash Here + 默认开始菜单 + VS Code（`--wait`）或 Nano + `main` + PATH 第二项 + bundled OpenSSH + Secure Channel + **Windows 单平台选 autocrlf=true，跨平台选 input** + MinTTY + Default pull + GCM + 文件系统缓存开、符号链接关。

2.3 首次启动与路径检查

安装完成后，勾选 **Launch Git Bash**，或从开始菜单打开 **Git Bash**。第一次启动会在用户目录下生成默认配置。先跑下面四条命令，确认“装的是哪一套、配置写到哪里、SSH 是否可用”。

```
$ git --version
git version 2.56.0.windows.4

$ git --exec-path
C:/Program Files/Git/mingw64/libexec/git-core

$ where.exe git
C:\Program Files\Git\cmd\git.exe

$ git config --list --show-origin
file:"C:/Program Files/Git/etc/gitconfig"  core.symlinks=false
file:"C:/Users/你的用户名/.gitconfig"       user.name=...
```

CHECK 01

版本号带 windows

`2.56.0.windows.4` 说明运行的是 Git for Windows 构建，而不是 WSL、MSYS2 或其他渠道的 Git。

CHECK 02

路径唯一

`where.exe git` 只应优先出现一条官方路径。多条不是错，但要知道第一条来自哪里。

把 Git Bash 加入 Windows Terminal

Windows Terminal 默认可能只有 PowerShell 和 CMD。打开“设置”→“添加新配置文件”→“复制 PowerShell 配置文件”，把命令行改成：

Windows Terminal profile

```
"commandline": "\"C:\\Program Files\\Git\\bin\\bash.exe\" -i -l",
"startingDirectory": "%USERPROFILE%",
"name": "Git Bash"
```

这里用 `bin\bash.exe` 而不是 `git-bash.exe`：前者让 Windows Terminal 接管标签页，后者会另开一个 MinTTY 窗口。启动后运行 `git --version` 验证。

长路径与中文文件名

Windows 的路径长度限制可能在深层 `node_modules` 或大型数据目录中触发 `Filename too long`。建议开启 Git 的长路径支持，并在系统策略中启用长路径：

```
# 当前用户生效，无需管理员
$ git config --global core.longpaths true

# 中文文件名不被转义显示（可选，但推荐）
$ git config --global core.quotepath false
```

系统级长路径还需要 `HKLM\SYSTEM\CurrentControlSet\Control\FileSystem\LongPathsEnabled=1`，可由管理员通过组策略或注册表设置。只改 Git 配置而不改系统策略，某些超过 260 字符的路径仍会失败。

> **PowerShell 里能运行 git，不代表 Git Bash 配置生效**
>
> Git 的系统配置在 `C:\Program Files\Git\etc\gitconfig`，用户配置在 `%USERPROFILE%\.gitconfig`，两者对 PowerShell 与 Git Bash 都生效。但 shell 的 PATH、别名、SSH agent 是各自独立的：在 Git Bash 里 `ssh-add` 的密钥，PowerShell 不一定能看到。第 6 章会按 shell 分别说明。

2.4 升级、静默安装与卸载

升级

三种方式按优先级选择：

方式	命令 / 操作	说明
Git 自带更新命令	`git update-git-for-windows`	在 Git Bash 中运行，检查并启动官方更新器；需要网络与管理员权限。
winget	`winget upgrade Git.Git`	适合脚本化；升级前先 `winget show Git.Git` 看目标版本。
官网安装器覆盖安装	下载新安装器，直接运行	最可控；会保留 `.gitconfig` 与已安装组件，按原选项覆盖。

表 2-4 Windows 升级方式。升级前关闭所有终端与 IDE，并确认没有未提交的改动。

升级后用 `git --version` 与 `where.exe git` 双重确认。若版本号没变，通常是旧进程占用、PATH 指向另一套 Git，或 winget 包尚未更新。不要反复安装同一个包，先定位实际运行的 `git.exe`。

企业静默安装

Git for Windows 安装器支持 Inno Setup 的静默参数。下面命令仅作为模板，部署前应在一台测试机上验证参数与版本：

```
PS> .\Git-2.56.0.4-64-bit.exe /VERYSILENT /NORESTART /NOCANCEL /SP- `
  /LOG="$env:TEMP\git-install.log"

# 安装后验证（新开的 PowerShell 窗口）
PS> git --version
PS> where.exe git
```

静默安装不会替你选择 PATH、换行符等策略。企业环境应通过安装器参数、组策略或首次登录脚本统一配置，并把配置写入版本可控的文档。日志文件是排查失败原因的第一手材料。

卸载与残留

打开“设置 → 应用 → 已安装的应用”，找到 **Git** 卸载。卸载器会移除程序目录、开始菜单项和它写入的系统 PATH，但**不会删除**以下内容：

```
C:\Users\你的用户名\.gitconfig        # 全局配置
C:\Users\你的用户名\.ssh\             # SSH 密钥与 known_hosts
C:\Users\你的用户名\AppData\Local\GitCredentialManager\  # GCM 缓存
C:\Users\你的用户名\AppData\Roaming\GitHub Desktop\       # 若装过 GitHub Desktop
```

如果只是重装或升级，保留这些目录可以省去重新配置；如果确定不再使用 Git，再手动删除。删除 `.ssh` 前确认里面没有其他服务（服务器、云主机）的密钥。卸载后重新打开终端，运行 `git --version` 应提示找不到命令；若仍能运行，检查 PATH 是否残留了其他渠道的 Git。

> **不要用“删除 Program Files 文件夹”来卸载**
>
> 这样会留下注册表项、PATH 条目和文件关联，后续重装可能出现“已安装”或路径冲突。始终优先使用“设置 → 应用”中的卸载器；便携版除外，它本来就是解压目录，删除前确认没有把配置写进该目录。

MACOS / APPLE SILICON & INTEL

03

macOS：Xcode CLT、官方包与 Homebrew

macOS 通常“自带 Git”，但自带的是 Apple 维护的版本，可能比官网旧。你需要先判断现有版本是否够用，再决定用系统 CLT、官方安装包，还是 Homebrew 管一个独立的新版本。

预计 18 分钟 | Apple Silicon / Intel | zsh 默认

这一章的顺序很重要：先检查，再安装。很多“macOS 装 Git 失败”的案例，其实是系统里已经有三套 Git，终端调用了旧的那一套。把路径和版本看清楚，比反复重装有效得多。

3.1 先检查系统自带的 Git

打开“终端”（Terminal），运行下面四条命令。第一条看版本，第二条看 Xcode Command Line Tools 是否已安装，第三条看所有可见的 Git 路径，第四条看当前 CPU 架构。

```
$ git --version
git version 2.39.5 (Apple Git-154)   # 示例，版本可能不同

$ xcode-select -p
/Library/Developer/CommandLineTools

$ which -a git
/usr/bin/git
/opt/homebrew/bin/git

$ uname -m
arm64        # Apple Silicon；Intel 机器显示 x86_64
```

如果 `git --version` 直接弹出“安装命令行开发者工具”的对话框，说明系统还没有可用的 Git，按提示安装即可。若命令正常返回但带 `(Apple Git-xxx)`，说明你用的是 Apple 自带版本；它通常足够完成日常提交与推送，但不一定包含最新 Git 功能。

现状	建议	原因
完全没有 Git，弹窗提示安装 CLT	安装 Xcode Command Line Tools	系统级、零额外维护，足够日常使用。
有 Apple Git，版本 2.39 左右	日常够用；需要新特性时换 Homebrew	Apple 版本偏保守，但与 macOS 集成最好。
项目要求 Git ≥ 2.40 / 2.50	Homebrew 安装最新稳定版	Homebrew 版本新，升级路径清晰。
公司统一分发 `.pkg`	按公司版本安装，不要混装	避免 PATH 中出现多个版本，审计也更简单。

表 3-1 macOS 现状与安装路线。不要为了“最新”而删除 `/usr/bin/git`，它属于系统组件。

安装 Xcode Command Line Tools

```
$ xcode-select --install
# 弹出软件更新窗口后，点击“安装”，等待下载完成

# 若提示许可未接受
$ sudo xcodebuild -license accept

# 验证
$ xcode-select -p
$ git --version
```

CLT 的安装包较大，下载时间取决于网络。安装完成后不需要重启，但已经打开的终端可能仍缓存旧路径；关闭终端重新打开，或执行 `hash -r` 刷新命令缓存。

3.2 三条安装路线

macOS 上常见的三种方式各有取舍。个人开发推荐 Homebrew；不想引入包管理器就用官方 `.pkg`；只是偶尔提交代码，系统 CLT 足够。

ROUTE A
Xcode CLT
系统自带渠道，安装最简单；版本随 macOS / Xcode 更新，通常不是最新 Git。适合轻度使用与公司统一环境。

ROUTE B
官方 .pkg
从 git-scm.com 下载签名安装包，版本新；安装到 `/usr/local/git`，需要管理员权限，升级要重新下载。

ROUTE C
Homebrew
开发者最常用；`brew install git` 一条命令，升级方便。需要先安装 Homebrew，并确保其路径优先于 `/usr/bin`。

路线 A：使用 Xcode CLT（最省事）

按 3.1 节的 `xcode-select --install` 安装即可。它的优点是系统集成度高、不会与 macOS 更新冲突；缺点是版本落后，某些新命令（例如较新的 `git maintenance` 子命令）可能不可用。若你只是 `clone`、`commit`、`push`，不必为此折腾。

路线 B：官方安装包（版本新、可控）

1. **下载**

   打开 `git-scm.com/download/mac`，下载最新的 macOS 安装包。注意区分 Intel 与 Apple Silicon；如果不确定，用 3.1 节的 `uname -m` 确认。

2. **校验并安装**

   双击 `.pkg`，按向导继续。若系统提示“无法验证开发者”，到“系统设置 → 隐私与安全性”中点“仍要打开”，不要用来源不明的脚本绕过 Gatekeeper。

3. **确认实际路径**

   安装器通常把文件放在 `/usr/local/git`，并通过 `/usr/local/bin` 暴露命令。运行 `which -a git` 与 `git --version`，确认新版本排在 `/usr/bin/git` 前面。

> **官方包与系统 Git 可以并存**
>
> 安装官方包不会删除 `/usr/bin/git`。终端到底调用哪一个，取决于 PATH 顺序。若安装后 `git --version` 仍是 Apple Git，检查 `which -a git` 的输出顺序，而不是急着重装。

路线 C：Homebrew（长期开发推荐）

Homebrew 把软件装在独立前缀：Apple Silicon 是 `/opt/homebrew`，Intel 是 `/usr/local`。它不会覆盖系统组件，卸载也干净。先确认是否已安装：

```
$ brew --version
Homebrew 4.x.x

# 若未安装，按官网命令安装（会要求输入密码并安装 CLT）
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# 安装 Git
$ brew install git

# 查看安装位置与版本
$ brew --prefix git
$ git --version
```

Apple Silicon 安装完 Homebrew 后，安装脚本通常会提示把下面一行加入 `~/.zshrc`。如果没有，手动补上：

~/.zshrc

```
# Apple Silicon
eval "$(/opt/homebrew/bin/brew shellenv)"

# Intel Mac 使用这一行
# eval "$(/usr/local/bin/brew shellenv)"
```

保存后执行 `source ~/.zshrc`（或新开终端），再运行 `hash -r` 与 `which -a git`。正确顺序应类似：

```
$ which -a git
/opt/homebrew/bin/git
/usr/bin/git

$ git --version
git version 2.56.0
```

Homebrew 安装后仍是旧版本?

先运行 `hash -r`，再检查 `~/.zshrc` 中 `brew shellenv` 是否在 PATH 设置之后；如果使用 fish、bash 或 IDE 内置终端，确认对应 shell 的配置文件也已加载。不要用 `sudo brew`，也不要把 Homebrew 的 Git 复制到 `/usr/bin`。

3.3 Shell、PATH 与 Gatekeeper

macOS 从 Catalina 起默认使用 `zsh`。配置文件通常是 `~/.zshrc`；如果你显式使用 bash，则是 `~/.bash_profile` 或 `~/.bashrc`。PATH 的顺序决定命令优先级：排在前面的目录先被找到。

路径	来源	典型用途
`/opt/homebrew/bin`	Apple Silicon Homebrew	Homebrew 安装的 git、ssh、工具链；应排在 `/usr/bin` 前。
`/usr/local/bin`	Intel Homebrew / 官方 pkg	官方 Git 安装包与 Intel Homebrew 的入口。
`/usr/bin`	macOS 系统	Apple Git；不要删除或替换。

表 3-2 macOS 常见 Git 路径。终端调用的是 PATH 中第一个匹配项。

临时调整当前终端（不写入配置文件）：

```
$ export PATH="/opt/homebrew/bin:$PATH"
$ hash -r
$ which git
```

Gatekeeper 拦截：如果双击 `.pkg` 提示“无法打开，因为 Apple 无法检查其是否包含恶意软件”，先确认下载来源是官方域名。然后到“系统设置 → 隐私与安全性”，在“安全性”区域点击“仍要打开”。不要习惯性地执行 `sudo spctl --master-disable` 或对下载文件批量 `xattr -d com.apple.quarantine`，那会削弱整台机器的防护。

配置 macOS 凭据助手

使用 HTTPS 访问 GitHub / GitLab 时，把凭据交给 macOS 钥匙串，避免每次输入令牌：

```
$ git config --global credential.helper osxkeychain
$ git config --global --get credential.helper
osxkeychain
```

第一次推送时输入的是**个人访问令牌（PAT）**，不是 GitHub 账号密码。令牌会被钥匙串保存；如需撤销，到“钥匙串访问”中搜索对应主机名删除条目，并在托管平台吊销令牌。

3.4 升级与卸载

安装方式	升级命令 / 操作	说明
Xcode CLT	`softwareupdate --all --install --force` 或系统更新	版本跟随系统；不保证拿到最新 Git。
官方 pkg	下载新版本 `.pkg` 覆盖安装	保留 `~/.gitconfig`；安装器会替换 `/usr/local/git`。
Homebrew	`brew upgrade git`	升级后 `hash -r`，再确认 `git --version`。

表 3-3 macOS 升级方式。升级前关闭 IDE 与终端中的 Git 进程。

卸载

- **Homebrew**：`brew uninstall git`。若要连同旧版本缓存一起清理，再运行 `brew cleanup git`。不会删除 `~/.gitconfig` 与 `~/.ssh`。
- **官方 pkg**：优先查看 `/usr/local/git/uninstall.sh` 是否存在；存在则运行 `sudo /usr/local/git/uninstall.sh`。不同版本的脚本位置可能不同，先 `ls /usr/local/git` 确认，不要直接删除整个 `/usr/local`。
- **Xcode CLT**：不建议单独卸载。它是系统开发工具的一部分，删除可能影响编译工具与其他软件。

卸载后执行 `hash -r` 与 `which -a git`。如果仍能运行 Git，说明还有另一套安装；按路径逐一定位。若 `/usr/bin/git` 仍存在，那是正常的系统 Git，不需要处理。

> **不要在 macOS 上“把 Git 装进系统目录”**
>
> 把 Homebrew 的二进制文件复制到 `/usr/bin`、或修改系统完整性保护（SIP）来替换 Apple Git，都会让系统更新与安全机制变得不可预测。正确做法是调整 PATH，让用户级安装优先。

LINUX / PACKAGES & SOURCE

04

Linux：包管理器与源码编译

Linux 上安装 Git 通常只有一条命令。真正需要判断的是：发行版仓库的版本够不够新、要不要引入第三方 PPA、以及什么时候值得从源码编译。顺序应当是“先用仓库，再考虑升级”，而不是一上来就编译。

预计 20 分钟 | 需要 sudo | Debian / Fedora / Arch / Alpine

服务器与企业环境优先使用发行版仓库：它有安全更新、可审计、能用配置管理工具批量部署。只有当你明确需要某个新版本或特定编译选项时，才使用 PPA 或源码。无论哪种方式，最后都要用 `which -a git` 确认实际调用的二进制。

4.1 发行版命令速查

先更新包索引，再安装。下表命令默认使用 `sudo`；容器内若已是 root，可去掉 `sudo`。

发行版	安装命令	验证
Debian / Ubuntu	`sudo apt update && sudo apt install -y git`	`git --version`
Fedora	`sudo dnf install -y git`	`git --version`
RHEL / Rocky / AlmaLinux	`sudo dnf install -y git`	`git --version`
Arch / Manjaro	`sudo pacman -S --needed git`	`git --version`
openSUSE	`sudo zypper install -y git`	`git --version`
Alpine	`sudo apk add git`	`git --version`
Amazon Linux 2023	`sudo dnf install -y git`	`git --version`

表 4-1 常见发行版的 Git 安装命令。包名统一为 `git`，但版本由发行版决定。

Debian / Ubuntu 的完整流程

```
$ sudo apt update
$ sudo apt install -y git
$ git --version
git version 2.43.0

# 查看仓库中的候选版本与来源
$ apt policy git
$ apt-cache show git | grep -E '^(Version|Homepage):'
```

如果 `apt policy` 显示候选版本来自 `noble/main` 之类的官方仓库，说明你用的是发行版维护的版本。它可能不是最新，但经过该发行版的兼容性测试。服务器上优先保留这种稳定性。

> **为什么 Linux 仓库里的 Git 常常偏旧?**
>
> 发行版会冻结主版本，只回移植安全修复与关键缺陷。Ubuntu 24.04 的 Git 可能停留在 2.43，而官网已经到 2.56。这不代表它“有问题”，而是稳定版策略。若项目明确要求新版本，再考虑 PPA 或源码。

4.2 Ubuntu 官方 PPA 与源码编译

Ubuntu：使用 git-core PPA

Git 项目维护了一个 Ubuntu PPA，适合需要较新稳定版、又不想自己编译的用户。它只适用于 Ubuntu，不适用于 Debian。引入第三方仓库前，确认公司安全策略允许。

```
$ sudo add-apt-repository ppa:git-core/ppa -y
$ sudo apt update
$ sudo apt install -y git
$ git --version
$ apt policy git
```

升级后若版本没变，检查 `apt policy git` 的候选来源是否已切换到 PPA。卸载 PPA 不会自动降级已安装的 Git；需要恢复发行版版本时，先移除 PPA，再 `sudo apt install --reinstall git`，具体版本以 `apt policy` 为准。

源码编译：需要精确控制时再用

源码编译适合以下场景：需要固定到某个 Git 版本、需要启用/禁用特定特性、或发行版仓库过旧且没有可信 PPA。缺点是不会自动更新，依赖与卸载都要自己管理。

1. **安装编译依赖（Debian / Ubuntu）**

   `sudo apt install -y build-essential libssl-dev libcurl4-openssl-dev libexpat1-dev gettext zlib1g-dev`

   不同发行版的包名不同；缺少依赖时 `make` 会在配置阶段报错。

2 **下载并校验源码**

从 `mirrors.edge.kernel.org/pub/software/scm/git/` 下载 `git-2.56.0.tar.xz`。用 `sha256sum` 计算摘要，并与发布公告中的值对照。

3 **编译并安装到 /usr/local**

使用 `prefix=/usr/local`，不要覆盖 `/usr/bin/git`。安装后用 `hash -r` 刷新命令缓存。

```
$ curl -LO https://mirrors.edge.kernel.org/pub/software/scm/git/git-2.56.0.tar.xz
$ sha256sum git-2.56.0.tar.xz
# 与官方发布公告中的 SHA-256 对照，确认一致再继续

$ tar -xf git-2.56.0.tar.xz
$ cd git-2.56.0
$ make prefix=/usr/local all
$ sudo make prefix=/usr/local install

$ hash -r
$ /usr/local/bin/git --version
$ which -a git
```

`make all` 只构建 Git 本体与常用工具；文档需要额外的 AsciiDoc 工具链，普通安装不必开启。安装后，`/usr/local/bin` 通常已在 PATH 中，且优先于 `/usr/bin`；如果没有，把 `/usr/local/bin` 加到 PATH 前面，或在脚本中使用绝对路径。

> **源码安装不会自动打补丁**
>
> 通过源码安装的 Git 不会收到 `apt upgrade` 的自动更新。你需要自己关注安全公告、重新下载与编译。生产服务器如果没有人负责这项维护，优先使用发行版仓库或官方 PPA。

4.3 验证、升级与卸载

安装完成后，至少运行下面四条命令。重点不是“有输出”，而是确认版本、路径与配置来源一致。

```
$ git --version
$ git --exec-path
$ which -a git
$ git config --list --show-origin --show-scope
```

输出异常	可能原因	处理
`git: command not found`	未安装，或 PATH 不包含安装目录	重新安装；用 `dpkg -L git` / `rpm -ql git` 查路径。
版本比预期旧	调用了 `/usr/bin/git`，而非 `/usr/local/bin/git`	`type -a git` 看顺序；调整 PATH 或删除重复安装。
版本比预期新，但命令缺失	多个 Git 混装，插件路径不匹配	统一到一个前缀；检查 `git --exec-path`。

表 4-2 Linux 上最常见的安装后异常。先定位路径，再决定是否重装。

升级

- **apt:** `sudo apt update && sudo apt upgrade git`。只升级 Git 时用后者；不要在没有备份的情况下执行全系统升级。
- **dnf:** `sudo dnf upgrade git`；**pacman:** `sudo pacman -Syu git`；**zypper:** `sudo zypper update git`；**apk:** `sudo apk upgrade git`。
- **源码：**下载新版本，重复编译与安装步骤；旧版本会被 `make install` 覆盖到同一前缀。

卸载

```
# Debian / Ubuntu
$ sudo apt remove git
$ sudo apt autoremove

# 源码安装（必须在源码目录中执行）
$ cd git-2.56.0
$ sudo make prefix=/usr/local uninstall

# 移除 PPA（可选）
$ sudo add-apt-repository --remove ppa:git-core/ppa -y
```

卸载不会删除 `~/.gitconfig` 与 `~/.ssh`。如果源码目录已经被删除，`make uninstall` 就无法执行；此时只能按安装清单手动删除 `/usr/local/bin/git*`、`/usr/local/libexec/git-core` 等路径，操作前先用 `make -n install` 或安装日志确认文件列表。

4.4 容器、WSL 与最小化环境

Docker / CI 镜像

精简镜像通常没有 Git，也缺少 SSH 客户端与 CA 证书。下面是一个可直接用于 Debian/Ubuntu 基础镜像的安装片段：

```Dockerfile
FROM debian:bookworm-slim

RUN apt-get update \
    && apt-get install -y --no-install-recommends \
       git openssh-client ca-certificates \
    && rm -rf /var/lib/apt/lists/*

# 验证
RUN git --version && ssh -V
```

Alpine 镜像使用 `apk add --no-cache git openssh-client ca-certificates`。CI 中常见 `detected dubious ownership`，是因为容器以 root 运行、仓库目录属于另一个 UID；不要全局关闭安全检查，按第 9.4 节把具体目录加入 `safe.directory`。

WSL2

在 WSL 的 Linux 发行版里单独安装 Git，不要把 Windows 的 `git.exe` 当成 WSL 的默认 Git。两者路径、换行符、凭据与 SSH agent 都不同。推荐把仓库放在 Linux 文件系统（如 `~/projects`），而不是 `/mnt/c`：跨文件系统访问会显著变慢，也容易触发权限与换行符问题。

```
$ sudo apt update
$ sudo apt install -y git
$ git config --global core.autocrlf input
$ git config --global init.defaultBranch main
```

> **最小化服务器的三项配套**
>
> 除了 Git 本身，确认已安装 `openssh-client`（SSH 远程）、`ca-certificates`（HTTPS 校验）与 `tzdata`（提交时间显示）。只装 Git 而不装 CA 证书，会在 HTTPS 克隆时报证书错误。

CONFIGURATION / FIRST RUN

05

安装后的第一轮配置

安装器只解决了“程序在哪”，配置才决定“提交是谁的、文件会不会被改写、推送要不要反复输密码”。这一章给出一次到位的配置顺序，并解释每一项在什么情况下需要改。

预计 20 分钟 | 跨平台通用 | 建议逐条执行

Git 的配置有层级：越靠近仓库的配置优先级越高。很多人“明明设置了全局邮箱，提交却还是旧邮箱”，原因往往是仓库本地配置覆盖了全局。先理解层级，再动手改。

5.1 配置层级与身份信息

层级	命令	文件位置	作用范围
system	`git config --system`	Windows: `C:\Program Files\Git\etc\gitconfig` macOS/Linux: `/etc/gitconfig`	整台机器，需管理员权限。
global	`git config --global`	`~/.gitconfig` 或 `~/.config/git/config`	当前用户的所有仓库。
local	`git config --local`	`.git/config`	当前仓库，优先级最高。

表 5-1 配置层级。优先级：local > global > system；命令行 `-c` 只对当前命令生效，优先级最高。

查看当前生效的配置与来源，这是排查“配置没生效”的第一条命令：

```
$ git config --list --show-origin --show-scope
global  file:C:/Users/you/.gitconfig  user.name=Zhang San
global  file:C:/Users/you/.gitconfig  user.email=zhangsan@example.com
local   file:.git/config              user.email=work@company.com
```

设置姓名与邮箱

Git 会把 `user.name` 与 `user.email` 写进每一个提交。邮箱建议使用你托管平台已验证的地址；否则 GitHub / GitLab 可能无法把提交归属到你的账号。

```
$ git config --global user.name "你的姓名"
$ git config --global user.email "you@example.com"

$ git config --global --get user.name
$ git config --global --get user.email
```

PRIVACY

不想公开真实邮箱?

GitHub 提供 `ID+username@users.noreply.github.com` 形式的专用邮箱，可在 Settings → Emails 中开启“Keep my email addresses private”并复制。它仍能把提交归属到账号。

WORK / PERSONAL

公司与个人身份分开

在仓库内执行 `git config user.email work@company.com`，只覆盖当前仓库；不要为了公司项目改全局身份。

如果希望 Git 在缺少身份时直接报错，而不是用系统默认值凑一个提交，可以开启：

```
$ git config --global user.useConfigOnly true
```

5.2 默认分支、编辑器与换行符

默认分支

新仓库的初始分支由 `init.defaultBranch` 决定。设为 `main` 可以与 GitHub / GitLab 的新仓库默认值保持一致；如果团队仍使用 `master`，跟随团队，不要制造两套命名。

```
$ git config --global init.defaultBranch main
$ git config --global --get init.defaultBranch
main
```

这只影响以后新建的仓库，不会重命名已有仓库的分支。已有仓库要改默认分支，用 `git branch -M main`，再到托管平台修改默认分支设置。

编辑器

不带 `-m` 的 `git commit`、交互式变基、合并提交信息都会打开编辑器。选择你熟悉的即可，关键是命令必须“阻塞”到文件关闭：

```
# VS Code（需已安装 code 命令）
$ git config --global core.editor "code --wait"

# Nano / Vim
$ git config --global core.editor nano
$ git config --global core.editor vim

# 验证：会打开编辑器，保存并关闭后返回终端
$ git config --global --edit
```

Windows 上若 `code` 不在 PATH，可在 VS Code 中执行“Shell Command: Install 'code' command in PATH”，或写完整路径。macOS 的 VS Code 也需要先安装 `code` 命令。不要用 `start`、`open` 这类立即返回的命令作为编辑器。

换行符：用一张表定下来

Git 的 `core.autocrlf` 与本机操作系统、团队平台有关。没有跨平台需求时，按下面选择；有跨平台需求时，优先把规则写进仓库的 `.gitattributes`。

你的环境	推荐配置	说明
Windows 单平台，团队都用 Windows	`core.autocrlf=true`	检出 CRLF，提交 LF；对旧 Windows 工具最友好。
Windows + Linux/macOS 混合	`core.autocrlf=input`	检出保持原样，提交统一 LF；配合 `.gitattributes` 最稳。
macOS / Linux	`core.autocrlf=input`	避免把 CRLF 提交进仓库；若仓库已有 `.gitattributes`，也可设为 `false`。
仓库已精确配置 `.gitattributes`	`core.autocrlf=false`	让仓库规则说了算，避免本机配置与仓库规则互相打架。

表 5-2 换行符配置。已有仓库不要直接批量转换，先用 `git status` 与 `git diff` 检查影响。

```
$ git config --global core.autocrlf input
$ git config --global --get core.autocrlf

# 可选：发现潜在的换行符问题时给出警告
$ git config --global core.safecrlf warn
```

如果仓库已有大量文件被意外改写，正确顺序是：先在分支上添加 `.gitattributes`，再执行 `git add --renormalize .`，检查 `git diff --cached` 后单独提交“统一换行符”。不要把格式化与业务修改混在同一个提交里。

中文文件名与编码

默认情况下，Git 会把非 ASCII 文件名转义成八进制显示，终端里看到一堆 `\344\270\255`。关闭转义后更易读：

```
$ git config --global core.quotepath false
$ git config --global i18n.commitEncoding utf-8
$ git config --global i18n.logOutputEncoding utf-8
```

5.3 凭据、代理与常用偏好

凭据助手

HTTPS 推送需要认证。Git 不会明文保存密码，而是调用系统凭据助手。按平台选择：

平台	推荐配置	说明
Windows	`credential.helper manager`	Git Credential Manager，支持 OAuth 与双因素认证。
macOS	`credential.helper osxkeychain`	存入系统钥匙串，随用户登录解锁。
Linux 桌面	`credential.helper libsecret`	需要系统提供 `git-credential-libsecret` 与 Secret Service。
Linux 服务器 / 容器	`credential.helper cache`	只缓存一段时间，适合无桌面的临时环境；不要缓存过久。

表 5-3 凭据助手。SSH 方式不需要凭据助手；两种方式可以并存。

```
$ git config --global credential.helper manager   # Windows
$ git config --global credential.helper osxkeychain # macOS
$ git config --global --get credential.helper
```

第一次 HTTPS 推送时，GitHub 会要求浏览器登录或输入 Personal Access Token；GitLab / Gitee 类似。**账号密码不能用于 GitHub 的 Git 操作**，必须使用令牌或 SSH。令牌权限与有效期在第 6.5 节说明。

代理

公司网络或本地代理软件可能需要显式配置 HTTP/HTTPS 代理。先确认代理地址与端口，再写入 Git 配置；不要从不明来源复制代理脚本。

```
# HTTP 代理（常见本地端口 7890 / 1080）
$ git config --global http.proxy http://127.0.0.1:7890
$ git config --global https.proxy http://127.0.0.1:7890

# SOCKS5
$ git config --global http.proxy socks5h://127.0.0.1:7890
$ git config --global https.proxy socks5h://127.0.0.1:7890

# 查看 / 取消
$ git config --global --get http.proxy
$ git config --global --unset http.proxy
$ git config --global --unset https.proxy
```

配置后测试：`git ls-remote https://github.com/git/git.git HEAD`。若仍然超时，用 `GIT_CURL_VERBOSE=1` 查看连接过程。系统环境变量 `HTTPS_PROXY` 也会影响 Git；Git 配置通常优先于环境变量，排查时两者都要看。

企业证书与 TLS

公司网络可能使用自签根证书做 TLS 检查。正确做法是把企业根证书配置给 Git，而不是关闭校验：

```
$ git config --global http.sslCAInfo "C:/certs/company-root-ca.pem"
$ git config --global --get http.sslCAInfo
```

Windows 使用 Secure Channel 后端时，通常直接信任系统证书库，不需要额外配置。只有使用 OpenSSL 后端、或证书不在系统库中时，才需要 `http.sslCAInfo`。永远不要用 `http.sslVerify=false` 作为长期方案。

值得一次性设置的偏好

配置	推荐值	作用
`fetch.prune`	`true`	拉取时清理远程已删除的分支引用，避免本地残留。
`push.default`	`simple`	只推送当前分支到同名远程分支，行为最可预测。
`push.autoSetupRemote`	`true`	首次推送自动建立 upstream，少写一次 `-u`（Git 2.37+）。
`rerere.enabled`	`true`	记住冲突解决方式，重复合并时自动复用。
`color.ui`	`auto`	终端支持颜色时高亮 diff / status。
`diff.algorithm`	`histogram`	某些代码结构下 diff 更清晰；团队无统一要求时可选。

表 5-4 常用偏好。不要盲目复制别人的全部配置；每项都应有明确理由。

```
$ git config --global fetch.prune true
$ git config --global push.default simple
$ git config --global push.autoSetupRemote true
$ git config --global rerere.enabled true
$ git config --global color.ui auto
```

别名：少打字，但别隐藏风险

```
$ git config --global alias.st "status -sb"
$ git config --global alias.lg "log --oneline --graph --decorate --all"
$ git config --global alias.last "log -1 --stat"

$ git st
$ git lg
```

不建议把 `reset --hard`、`push --force` 这类破坏性命令做成过短的别名，尤其是容易误触的单个字母。别名应减少重复，而不是降低警惕。

5.4 配置文件备份与迁移

换电脑时，最值得带走的是 `~/.gitconfig`、`~/.ssh` 与项目清单；最不该带走的是访问令牌的明文。配置文件本身通常不含密码，但可能包含代理地址、公司邮箱与内部域名，备份时仍要注意存放位置。

```
# macOS / Linux：带日期备份
$ cp ~/.gitconfig ~/gitconfig.backup-$(date +%F)

# Windows PowerShell
PS> Copy-Item "$HOME\.gitconfig" "$HOME\gitconfig.backup-$(Get-Date -Format yyyy-MM-dd)"
```

用 includeIf 区分公司与个人身份

如果你的公司仓库都在 `~/work/` 下，可以让该目录自动加载另一份配置，而不必每次手动切换邮箱：

~/.gitconfig

```
[user]
    name = Zhang San
    email = personal@example.com

[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work
```

~/.gitconfig-work

```
[user]
    name = Zhang San
    email = zhangsan@company.com
```

目录匹配规则对大小写敏感；Windows 上可用 `gitdir/i:` 忽略大小写，路径写成 `C:/work/`。修改后用 `git config --show-origin --get user.email` 在目标仓库中验证。不要把令牌、密码写进这些文件。

迁移检查单

新电脑上恢复 `.gitconfig` 后，逐项确认：`user.name` / `user.email`、`core.autocrlf`、`credential.helper`、代理设置、`includeIf` 路径。SSH 私钥要单独安全迁移，权限保持 `600`；不要把 `.ssh` 放进公开网盘。

5.5 配置验收：四条命令

配置是否“写进去了”，不能只看命令有没有报错。用下面四条命令分别验证身份、来源、换行符与凭据；它们覆盖了后续 90% 的“配置不生效”问题。

检查	命令	通过标准
提交身份	`git var GIT_AUTHOR_IDENT`	输出“姓名 <邮箱> 时间”；邮箱与平台账号一致。
配置来源	`git config --show-origin user.email`	能看出邮箱来自 global 还是 local，避免仓库覆盖。
换行符	`git config --global core.autocrlf`	Windows 单平台 `true`；跨平台 `input`；空值表示未设置。
凭据助手	`git config --global credential.helper`	Windows `manager`；macOS `osxkeychain`；Linux `libsecret` 或 `cache`。

表 5-5 配置验收。若某条为空，先补配置，不要等到第一次 push 才发现。

WINDOWS

起点

`autocrlf=true`、`helper=manager`、编辑器 `code --wait`；跨平台项目改 `input`。

MACOS

起点

`autocrlf=input`、`helper=osxkeychain`；Homebrew Git 放在 PATH 前。

LINUX

起点

`autocrlf=input`、`helper=libsecret`（桌面）或 `cache`（服务器）。

配置只写一次，验证要每次换机都做

把 5.5 的四条命令加入新机器初始化脚本。配置文件的复制不等于配置生效：路径、include、环境变量与仓库 local 配置都可能改变最终结果。

AUTHENTICATION / SSH & HTTPS

06

SSH 与 HTTPS：安全连接远程仓库

远程仓库只认两种东西：密钥，或令牌。SSH 用一对密钥完成认证，HTTPS 用账号加令牌。选哪种不影响 Git 的本地能力，但会影响日常体验、企业网络兼容性与密钥管理成本。

预计 30 分钟 | 需要托管平台账号 | 含多账号配置

先把一个常见误解说清楚：SSH 密钥不是“给 Git 用的密码”，而是给远程服务器验证你身份用的。私钥留在本机，公钥上传到 GitHub / GitLab / Gitee。任何人拿到私钥都能冒充你，所以私钥永远不能发出去，也不能提交进仓库。

6.1 选 SSH 还是 HTTPS

维度	SSH	HTTPS
认证方式	本机私钥 + 平台上的公钥	账号 + Personal Access Token（PAT）
首次配置	生成密钥、上传公钥、配置 agent	配置凭据助手，浏览器登录或粘贴令牌
日常体验	配置好后通常无感，适合多仓库	GCM 可无感；无助手时要反复输入令牌
网络兼容性	默认 22 端口；公司网络可能封禁，可改走 443	走 443，通常更容易穿过代理与防火墙
多账号	通过 `~/.ssh/config` 的 Host 别名区分	通过不同凭据 / 用户名区分，容易串号
适用场景	长期开发、服务器、自动化	受限网络、临时环境、只想少配密钥

表 6-1 SSH 与 HTTPS 对比。两者可以同时使用；不必二选一“卸载”另一种。

选择建议

个人长期开发：SSH，配一次用很久。**公司电脑、网络限制严格：**HTTPS + GCM，或 SSH over 443。**CI / 服务器：**部署密钥（Deploy Key）或短期令牌，权限最小化。**多账号：**SSH 的 Host 别名最清晰，但必须写 `IdentitiesOnly yes`。

6.2 生成与保管 SSH 密钥

先检查是否已有密钥，避免覆盖：

```
$ ls -la ~/.ssh
id_ed25519      id_ed25519.pub      known_hosts
```

如果已有可用的 `id_ed25519`，可以直接使用；没有则生成。推荐 Ed25519：密钥短、速度快、安全性好。只有连接很旧的服务器时才用 RSA 4096。

```
# 默认文件名 ~/.ssh/id_ed25519
$ ssh-keygen -t ed25519 -C "you@example.com"

# 为 GitHub 单独生成一把，避免与其他服务共用
$ ssh-keygen -t ed25519 -C "github-personal" -f ~/.ssh/id_ed25519_github

# 旧服务器兼容（不推荐作为首选）
$ ssh-keygen -t rsa -b 4096 -C "legacy-server" -f ~/.ssh/id_rsa_legacy
```

执行时会询问保存路径与口令（passphrase）。**建议设置口令**：即使私钥文件泄露，攻击者仍需口令才能使用。口令由 `ssh-agent` 缓存，不会每次输入。直接回车留空也可以，但安全性更低，尤其是在共享电脑上。

PRIVATE

`id_ed25519`

私钥，权限必须严格限制。不要上传、不要发聊天工具、不要放进仓库或云笔记。泄露后立即在平台删除对应公钥并重新生成。

PUBLIC

`id_ed25519.pub`

公钥，可以安全地粘贴到 GitHub / GitLab / Gitee。内容以 `ssh-ed25519` 开头，末尾是注释，不含私密信息。

文件权限

OpenSSH 对权限很敏感。macOS / Linux / Git Bash 中执行：

```
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/id_ed25519 ~/.ssh/id_ed25519_github
$ chmod 644 ~/.ssh/id_ed25519_github.pub
$ chmod 600 ~/.ssh/config 2>/dev/null || true
```

Windows 上如果 SSH 仍报 `Bad permissions`，说明文件 ACL 允许了其他用户读取。可在 PowerShell（管理员）中用 `icacls` 移除继承并只授权当前用户；不要用“把整个用户目录设为所有人可读”来图省事。

6.3 ssh-agent 与多账号配置

`ssh-agent` 负责在会话中缓存解密后的私钥，避免每次连接都输入口令。不同系统的启动方式不同。

Windows（Git Bash）

```
$ eval "$(ssh-agent -s)"
Agent pid 12345
$ ssh-add ~/.ssh/id_ed25519_github
$ ssh-add -l
256 SHA256:... github-personal (ED25519)
```

Git Bash 的 agent 通常只在当前会话有效；关闭终端后需要重新添加。若你使用 Windows 自带的 OpenSSH，可启用系统服务 `ssh-agent`，让密钥在登录后持续可用；但这与 Git for Windows 自带的 `ssh.exe` 是两套程序，务必确认 Git 实际调用的是哪一套（`where.exe ssh`）。

macOS

```
$ eval "$(ssh-agent -s)"
$ ssh-add --apple-use-keychain ~/.ssh/id_ed25519_github
$ ssh-add -l
```

要让钥匙串记住口令，在 `~/.ssh/config` 中加入：

~/.ssh/config

```
Host *
    AddKeysToAgent yes
    UseKeychain yes
```

较新的 macOS 使用 `--apple-use-keychain`；旧版本可能是 `-K`。如果命令不存在，先看 `ssh-add -h` 的选项。

Linux

```
$ eval "$(ssh-agent -s)"
$ ssh-add ~/.ssh/id_ed25519_github
$ ssh-add -l
```

服务器上不要把 `eval "$(ssh-agent -s)"` 无条件写进 `.bashrc`，否则每次开子 shell 都会启动新 agent。需要持久化时，使用桌面环境的 keyring、systemd 用户服务，或只在交互式登录时启动一次。

多账号：用 Host 别名隔离

假设你有个人 GitHub 与公司 GitHub，两个账号不能共用同一把密钥（也可以共用，但隔离更安全）。编辑 `~/.ssh/config`：

~/.ssh/config

```
# 个人账号
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519_github
    IdentitiesOnly yes

# 公司账号：Host 是自定义别名，不是真实域名
Host github-work
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519_work
    IdentitiesOnly yes
```

公司仓库的远程地址要使用别名：

```
$ git clone git@github-work:company/repo.git
$ ssh -T git@github-work
Hi company-username! You've successfully authenticated...
```

`IdentitiesOnly yes` 很关键：它让 SSH 只尝试该 Host 指定的密钥，避免 agent 里一堆密钥轮流试，触发 `Too many authentication failures`。Host 块要放在 `Host *` 之前，否则通配配置可能覆盖。

6.4 添加公钥并测试连接

先复制**公钥**（`.pub` 结尾）的完整内容：

系统	复制命令	备注
Windows Git Bash	`clip < ~/.ssh/id_ed25519_github.pub`	复制到剪贴板。
Windows PowerShell	`Get-Content ~/.ssh/id_ed25519_github.pub | Set-Clipboard`	路径可用 `$HOME\.ssh\...`。
macOS	`pbcopy < ~/.ssh/id_ed25519_github.pub`	系统自带。
Linux 桌面	`xclip -selection clipboard < ~/.ssh/id_ed25519_github.pub`	需安装 `xclip`；服务器上直接 `cat` 后手动复制。

表 6-2 复制公钥。确认复制的是 `.pub` 文件，而不是私钥。

添加到托管平台

- **GitHub**：Settings → SSH and GPG keys → New SSH key。Title 写设备名，Key type 选 Authentication Key，粘贴公钥。
- **GitLab**：Preferences → SSH Keys → Add new key。可设置过期时间；到期前记得轮换。
- **Gitee**：设置 → SSH 公钥 → 添加公钥。标题写清设备，便于以后撤销。
- **服务器**：把公钥追加到 `~/.ssh/authorized_keys`，权限 `600`，目录 `700`。

测试

```
$ ssh -T git@github.com
Hi username! You've successfully authenticated, but GitHub does not provide shell access.

$ ssh -T git@gitlab.com
Welcome to GitLab, @username!
```

看到问候语即成功。**GitHub 的测试命令可能以退出码 1 结束**，因为它不提供 shell，这是正常现象；不要只看 `$?`，要看输出。若失败，用 `ssh -vT git@github.com` 查看握手、密钥加载与认证过程。

首次连接的主机指纹

第一次连接会提示 `The authenticity of host ... can't be established`，并要求确认指纹。应到平台官方文档核对指纹，而不是盲目输入 `yes`；更不要设置 `StrictHostKeyChecking=no` 永久跳过校验。如果指纹突然变化，先停下，确认是否为公司代理或中间人。

6.5 HTTPS 令牌与常见认证错误

GitHub 从 2021 年起不再接受账号密码进行 Git 操作。HTTPS 推送时，用户名填 GitHub 用户名，密码位置填 **Personal Access Token**。令牌相当于密码，必须按最小权限、短有效期管理。

平台	令牌类型	最小权限建议
GitHub	Fine-grained token	仅选目标仓库；Contents 读写；Metadata 只读。不要勾选全仓库管理权限。
GitHub	Classic token	`repo` 权限过大，除非必要，优先 Fine-grained。
GitLab	Personal Access Token	`read_repository` + `write_repository`；设置过期时间。
Gitee	私人令牌	按提示选择仓库读写；不要授予账号级全权限。

表 6-3 HTTPS 令牌。令牌只显示一次，丢失后重新生成，不要试图找回。

不要把令牌写进远程 URL

下面这种写法会把令牌明文保存在 `.git/config`，并可能出现在终端历史、日志与截图中：

```
# 不要这样做
https://ghp_xxxxxxxxxxxxxxxxxxxx@github.com/user/repo.git
```

正确做法是让凭据助手保存，远程 URL 保持干净：`https://github.com/user/repo.git`。如果已经写入，立即在平台吊销令牌，并执行 `git remote set-url origin https://github.com/user/repo.git`。

认证错误速查

报错	原因	处理
`Permission denied (publickey)`	服务器没拿到可用公钥	`ssh -vT` 看是否加载了正确私钥；`ssh-add -l`；确认公钥已上传、Host 配置匹配。
`Support for password authentication was removed`	用账号密码推 GitHub	改用 PAT 或 SSH；配置 GCM。
`remote: Invalid username or password`	令牌错误、过期或权限不足	重新生成令牌，确认仓库读写权限；清除旧凭据后重试。

报错	原因	处理
`Permission to repo.git denied to user`	登录的账号没有该仓库权限	确认账号、组织授权、SSO 是否已授权；多账号时检查当前凭据。
`SSL certificate problem`	企业自签证书 / 代理拦截	配置企业根证书 `http.sslCAInfo`；不要关闭校验。
`Connection timed out`（22 端口）	公司网络封禁 SSH 22	配置 SSH over 443，或改用 HTTPS。
`REMOTE HOST IDENTIFICATION HAS CHANGED`	主机指纹变化，可能是中间人	先核实原因；确认后再删除 `known_hosts` 中对应条目，不要直接清空整个文件。
`Too many authentication failures`	agent 里密钥太多，逐个尝试被拒	在 Host 块加 `IdentitiesOnly yes`；只加载需要的密钥。
`Bad owner or permissions`	私钥或 config 权限过宽	`chmod 600`；Windows 用 `icacls` 限制 ACL。

表 6-4 SSH / HTTPS 常见认证错误。先看完整报错，再对照处理，不要同时改五个地方。

SSH over 443（绕过 22 端口封锁）

GitHub 支持通过 `ssh.github.com:443` 建立 SSH 连接。把下面配置放在 `~/.ssh/config` 中：

~/.ssh/config

```
Host github.com
    HostName ssh.github.com
    Port 443
    User git
    IdentityFile ~/.ssh/id_ed25519_github
    IdentitiesOnly yes
```

```
$ ssh -T -p 443 git@ssh.github.com
$ ssh -T git@github.com
```

如果公司代理只允许 HTTP/HTTPS，SSH over 443 仍可能被拦截；此时改用 HTTPS + GCM，或让网络管理员放行。不要为了绕过公司安全策略而使用未经批准的隧道工具。

FIRST REPOSITORY / HANDS ON

07

第一个仓库：从 init 到 push

现在把前面的配置串起来：创建一个本地仓库、完成第一次提交、连接远程、推送成功。这条链路跑通，Git 才算真正可用；之后再学分支、撤销与冲突，才有稳定的地基。

预计 25 分钟 | 需要远程账号 | 可跟着做

下面的命令以 Git Bash / macOS Terminal / Linux 为准。Windows PowerShell 也能运行 Git，但创建文本文件时的编码行为不同；本节的示例文件用 Git Bash 的 `printf` 生成，避免 PowerShell 5.1 写出 UTF-16。

7.1 创建仓库并完成第一次提交

先确认身份已配置：`git config --global user.name` 与 `git config --global user.email` 都有输出。然后在一个空目录中初始化：

```
$ mkdir git-demo
$ cd git-demo
$ git init
Initialized empty Git repository in .../git-demo/.git/

$ git status
On branch main
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

`git init` 只创建 `.git` 目录，不会自动提交任何文件。接下来创建一个 README，把它加入暂存区并提交：

```
$ printf '# Git Demo\n\nMy first repository.\n' > README.md
$ git status
Untracked files:
    README.md

$ git add README.md
$ git status
Changes to be committed:
    new file:   README.md

$ git commit -m "docs: add README"
[main (root-commit) 1a2b3c4] docs: add README
 1 file changed, 3 insertions(+)
```

STATUS

看懂三种状态

Untracked：从未被 Git 记录。**Changes not staged**：已跟踪但修改未暂存。**Changes to be committed**：已放入暂存区，等待提交。

COMMIT

提交信息怎么写

第一行用一句话说明“做了什么”，不超过 50 个字符；需要时空一行再写原因。团队若使用 Conventional Commits，遵循 `feat:`、`fix:`、`docs:` 前缀。

查看历史与本次改动：

```
$ git log --oneline --decorate
1a2b3c4 (HEAD -> main) docs: add README

$ git show --stat HEAD
$ git diff HEAD~1..HEAD
```

先小步提交，再谈“完美历史”

第一次提交不需要包含整个项目。先让 `init → add → commit` 跑通，确认用户名、邮箱、换行符都没有问题；再逐步加入代码与 `.gitignore`。一个可运行的仓库，比一个“结构完美但推不上去”的仓库更有价值。

7.2 连接远程仓库与克隆

在托管平台创建空仓库

在 GitHub / GitLab / Gitee 新建仓库时，**不要**勾选“添加 README”“添加 .gitignore”“选择许可证”。空仓库不会产生初始提交，可以避免第一次推送时的“远程已有提交”冲突。创建后复制仓库地址：SSH 形如 `git@github.com:USERNAME/git-demo.git`，HTTPS 形如 `https://github.com/USERNAME/git-demo.git`。

```
$ git remote add origin git@github.com:USERNAME/git-demo.git
$ git remote -v
origin  git@github.com:USERNAME/git-demo.git (fetch)
origin  git@github.com:USERNAME/git-demo.git (push)

$ git branch -M main
$ git push -u origin main
branch 'main' set up to track 'origin/main'.
```

`-u` 把本地 `main` 与远程 `origin/main` 建立跟踪关系；之后直接 `git push` / `git pull` 即可。若已经配置了 `push.autoSetupRemote=true`，首次推送可以省略 `-u`，但显式写出来更清楚。

报错	原因	处理
`remote origin already exists`	已经添加过 `origin`	`git remote set-url origin <新地址>`，不要重复 add。
`src refspec main does not match any`	本地还没有提交，或分支名不是 main	先 `git commit`；用 `git branch --show-current` 看当前分支。
`Updates were rejected because the remote contains work...`	远程仓库已有提交（例如初始化了 README）	先 `git pull --rebase origin main`，解决冲突后再 push；或改为 clone 远程仓库。
`Permission denied (publickey)`	SSH 未配置好	回到第 6 章测试 `ssh -T git@github.com`，或临时改用 HTTPS。

表 7-1 第一次 push 的高频错误。先读完整报错，不要直接 `--force`。

克隆已有仓库

```
# SSH（推荐长期使用）
$ git clone git@github.com:OWNER/REPO.git

# HTTPS（受限网络 / 临时环境）
$ git clone https://github.com/OWNER/REPO.git

# 指定目录、分支、子模块
$ git clone git@github.com:OWNER/REPO.git my-folder
$ git clone --branch dev --single-branch git@github.com:OWNER/REPO.git
$ git clone --recurse-submodules git@github.com:OWNER/REPO.git
```

选项	用途与注意
`--depth 1`	浅克隆，只取最新一次提交；适合 CI，不适合需要完整历史的开发。
`--filter=blob:none`	部分克隆，按需下载文件内容；适合超大仓库，需要服务器支持。
`--branch <name>`	克隆指定分支；配合 `--single-branch` 减少引用。
`--recurse-submodules`	同时初始化子模块；漏掉会看到空目录。

表 7-2 克隆选项。普通开发默认完整克隆，遇到体积问题再考虑浅克隆或部分克隆。

已有本地项目，后来才建远程仓库

```
$ cd existing-project
$ git init
$ git add .
$ git commit -m "chore: initial import"
$ git remote add origin git@github.com:USERNAME/existing-project.git
$ git branch -M main
$ git push -u origin main
```

如果项目里已经有一堆构建产物，先写 `.gitignore` 再 `git add .`，不要先提交再删除。已经提交的大文件会留在历史里，清理需要重写历史，成本更高。

7.3 .gitignore 与分支协作

先忽略，再添加

`.gitignore` 只对“尚未被跟踪”的文件生效。已经被 Git 跟踪的文件，即使写进忽略规则也不会自动消失，需要 `git rm --cached`。

.gitignore

```
# 操作系统
.DS_Store
Thumbs.db
desktop.ini

# 编辑器 / IDE
.vscode/
.idea/
*.swp

# Python
__pycache__/
*.py[cod]
.venv/

# Node
node_modules/
dist/

# 密钥与本地配置
.env
.env.*
!.env.example
*.pem
*.key
```

```
$ git check-ignore -v .env
.gitignore:20:.env    .env

# 文件已被跟踪时，先从索引移除（保留本地文件）
$ git rm --cached .env
$ git commit -m "chore: stop tracking local env file"
```

忽略文件不能替代密钥轮换

如果密钥、令牌或 `.env` 已经提交并推送，仅仅删除文件不够：它们仍留在历史中。应立即吊销/轮换密钥，再评估是否需要使用 `git filter-repo` 清理历史。不要以为“删掉最新提交”就安全了。

分支：从创建到合并

```
$ git switch -c feature/login
Switched to a new branch 'feature/login'

# 修改文件后
$ git add .
$ git commit -m "feat: add login form"
$ git push -u origin feature/login

# 回到 main，合并（团队通常通过 PR/MR 完成）
$ git switch main
$ git pull --ff-only
$ git merge --no-ff feature/login
$ git push
$ git branch -d feature/login
```

`git switch` 与 `git checkout` 的区别：前者只用于切换分支，语义更清晰；后者还能恢复文件。新版本 Git 推荐用 `switch` 切换、`restore` 恢复文件，减少误操作。

团队协作通常不在本地直接合并，而是推送功能分支后在平台上发起 Pull Request / Merge Request。流程是：功能分支提交 → push → 平台发起 PR → 代码审查 → CI 通过 → 合并 → 删除远程分支。合并前先 `git pull --rebase origin main`，把功能分支变基到最新 main，减少冲突。

7.4 撤销、回退与冲突处理

Git 的“撤销”不是单一命令，而是按“改到哪一层”区分。先看表，再动手；涉及 `--hard` 与 `--force` 的操作，先确认没有未备份的改动。

场景	命令	说明
丢弃工作区某个文件的修改	`git restore <file>`	不可恢复；执行前确认不需要这些改动。
把文件移出暂存区	`git restore --staged <file>`	保留工作区内容，只取消 `add`。

场景	命令	说明
修改上一条提交信息	`git commit --amend`	只用于尚未推送的提交；会改写提交哈希。
撤销一条已推送的提交	`git revert <hash>`	生成一条反向提交，安全，适合共享分支。
回退一条本地提交，保留改动	`git reset --soft HEAD~1`	提交消失，改动回到暂存区；不要用于已推送历史。
彻底丢弃最近一次提交与改动	`git reset --hard HEAD~1`	**危险**；未提交内容会丢失。先用 `git reflog` 确认。
找回误操作的提交	`git reflog` + `git switch -c rescue <hash>`	reflog 记录 HEAD 移动；越早恢复越可靠。

表 7-3 常见撤销操作。共享分支优先 `revert`，不要用 `reset` 改写别人已经拉取的历史。

合并冲突：按四步处理

当两个分支修改了同一文件的同一区域，`git merge` 会停下并标记冲突：

```
$ git status
Unmerged paths:
    both modified:   src/app.py

$ git diff --name-only --diff-filter=U
```

打开冲突文件，会看到 `<<<<<<<`、`=======`、`>>>>>>>` 标记。手动编辑为最终内容，删除所有标记，然后：

```
$ git add src/app.py
$ git commit
# 或放弃这次合并
$ git merge --abort
```

变基冲突的处理类似，但命令不同：解决后 `git add`，再 `git rebase --continue`；想放弃用 `git rebase --abort`。`git rebase --skip` 会丢弃当前提交，除非你明确知道后果，不要使用。

> **冲突不是“文件坏了”**
>
> 冲突标记是 Git 在告诉你“这两段改动无法自动合并”。正确做法是理解两边意图，保留需要的内容，运行测试，再提交。不要为了快速消失而一律选择“当前分支”或“传入分支”，那可能静默丢掉别人的修复。

VERIFICATION / ACCEPTANCE

08

验收：确认安装真的可用

“能显示版本号”只是第一层。真正的验收要覆盖四件事：命令能找到、配置写对了、身份能归属、远程能认证。下面十项检查可以在五分钟内跑完，适合新机器交付与故障复现。

预计 12 分钟 | 全平台通用 | 建议保存为脚本

8.1 十项验收清单

#	检查项	命令	通过标准
1	版本可用	`git --version`	返回 2.40+；Windows 带 `windows`，macOS 可带 `Apple Git`。
2	路径唯一	`where.exe git` / `which -a git`	第一条指向你安装的版本；知道其余路径来自哪里。
3	执行路径正确	`git --exec-path`	路径与当前安装前缀一致，不指向已卸载目录。
4	配置来源清楚	`git config --list --show-origin --show-scope`	能看到 global / local 来源；没有意外覆盖。
5	身份已设置	`git config --global --get user.name` `git config --global --get user.email`	都有输出；邮箱是已验证地址或 noreply 地址。
6	默认分支正确	`git config --global --get init.defaultBranch`	返回 `main` 或团队约定的名称。
7	换行符符合约定	`git config --global --get core.autocrlf`	Windows 单平台 `true`；跨平台 `input`；有 .gitattributes 时可为 `false`。
8	凭据助手可用	`git config --global --get credential.helper`	Windows 为 `manager`，macOS 为 `osxkeychain`；只用 SSH 可为空。
9	SSH 已打通	`ssh -T git@github.com`	看到 “successfully authenticated”；退出码可能为 1。
10	远程可读取	`git ls-remote https://github.com/git/git.git HEAD`	输出一行 HEAD 哈希，退出码 0。

表 8-1 Git 安装十项验收。第 9 项只在使用 SSH 时执行；第 10 项同时验证 DNS、TLS 与代理。

把验收做成一条命令

macOS / Linux / Git Bash 可以把下面脚本保存为 `git-check.sh`，运行 `bash git-check.sh`。它只读取信息，不修改配置。

git-check.sh

```
#!/usr/bin/env bash
set -u

echo "== version =="
git --version

echo "== path =="
command -v git
git --exec-path

echo "== identity =="
git config --global --get user.name  || echo "MISSING user.name"
git config --global --get user.email || echo "MISSING user.email"

echo "== branch / eol / helper =="
git config --global --get init.defaultBranch || true
git config --global --get core.autocrlf || true
git config --global --get credential.helper || true

echo "== remote reachability =="
git ls-remote https://github.com/git/git.git HEAD
```

Windows PowerShell 版本：

git-check.ps1

```
git --version
where.exe git
git --exec-path
git config --global --get user.name
git config --global --get user.email
git config --global --get init.defaultBranch
git config --global --get core.autocrlf
git config --global --get credential.helper
git ls-remote https://github.com/git/git.git HEAD
```

8.2 诊断命令与正常输出

验收失败时，不要立刻重装。先收集下面四类信息，通常能在两分钟内定位到“路径、配置、网络、认证”中的一层。

命令	回答什么问题	怎么看
`git --version`	实际运行的是哪个版本	与安装包版本对比；Windows 带 `windows`。
`git --exec-path`	Git 的核心程序在哪	应位于当前安装目录；若指向旧版本，PATH 有问题。

`git var GIT_AUTHOR_IDENT`	提交会使用谁的身份	返回 `姓名 <邮箱> 时间`；比逐项查配置更直接。
`git config --show-origin --get user.email`	邮箱来自哪个文件	确认是 global 还是 local，避免“全局改了但仓库覆盖”。
`git ls-remote <url> HEAD`	远程是否可达、认证是否通过	输出哈希即成功；失败会给出 403、404、超时或证书错误。
`ssh -vT git@github.com`	SSH 用了哪把钥匙	看 `Offering public key` 与 `Server accepts key`；确认没有加载错密钥。
`GIT_TRACE=1 git fetch`	Git 执行了哪些子命令	适合排查钩子、凭据助手、SSH 命令调用。
`GIT_CURL_VERBOSE=1 git fetch`	HTTPS 连接细节	看 DNS、代理、TLS 握手、HTTP 状态码；输出很多，先看第一处错误。

表 8-2 诊断命令。把完整输出（隐去令牌与隐私）贴给同事或 Issue，比只说“push 失败”有效得多。

一个最小的本地提交测试

如果远程认证暂时不通，至少先验证本地仓库能完成提交。这能区分“Git 没装好”与“只是网络/认证问题”。

```
$ mkdir git-smoke-test && cd git-smoke-test
$ git init
$ printf 'hello\n' > hello.txt
$ git add hello.txt
$ git commit -m "test: verify local git"
$ git log --oneline -1
a1b2c3d test: verify local git
```

8.3 网络与代理连通性测试

Git 的 HTTPS 走 libcurl，SSH 走 OpenSSH；两者失败原因不同，要分开测。先测 DNS 与 443 端口，再测 Git 协议。

HTTPS

443 端口与证书

`curl -I https://github.com` 应返回 `HTTP/2 200` 或 301。若卡住，查 DNS、代理、企业证书。

SSH

22 端口与密钥

`ssh -T git@github.com` 测认证；`ssh -vT` 看握手。超时多半是 22 被封锁，改用 443。

```
# 1. DNS 与 HTTPS（macOS / Linux / Git Bash）
$ curl -I https://github.com
$ curl -I https://gitlab.com

# 2. Git 协议层（不需要登录）
$ git ls-remote https://github.com/git/git.git HEAD

# 3. 代理环境变量是否生效
$ env | grep -i proxy

# 4. SSH 认证
$ ssh -T git@github.com
```

```
# Windows PowerShell
PS> Test-NetConnection github.com -Port 443
PS> Invoke-WebRequest -Method Head https://github.com
PS> git ls-remote https://github.com/git/git.git HEAD
PS> ssh -T git@github.com
```

现象	最可能的原因	下一步
`Could not resolve host`	DNS 或网络未连通	检查 Wi-Fi / VPN；`nslookup github.com`。
连接超时，但浏览器能开	终端未走系统代理	配置 `http.proxy`，或检查 `HTTPS_PROXY`。
`SSL certificate problem`	企业自签根证书	配置 `http.sslCAInfo`，不要关闭校验。
HTTP 403 / 404	令牌权限不足，或仓库地址/权限错误	检查 PAT 权限、组织 SSO、仓库是否存在。
SSH 22 超时	公司网络封禁 22	用 SSH over 443，或改用 HTTPS。

表 8-3 网络层排查顺序：DNS → 端口 → 代理 → TLS → 认证。

验收通过后，保存一份“机器基线”

把 `git --version`、`git --exec-path`、`git config --global --list` 的输出保存到团队 Wiki 或机器初始化文档。下次出现“以前能用，现在不行”，可以直接对比差异，而不是从零猜。

TROUBLESHOOTING / ERROR INDEX

09

常见问题：从报错到修复

这一章按“安装与 PATH → 提交与推送 → 认证与网络 → 权限与其他”分组。每条先给最可能的原因，再给最小修复步骤。不要一次改五处；一次只动一个变量，才能知道是哪一步生效。

按需查阅 | 含 23 个高频问题 | 命令可直接复制

9.1 安装与 PATH 类问题

01 PowerShell / CMD 提示“无法将 git 项识别为 cmdlet、函数……”

Git 没进 PATH，或当前终端还是安装前的环境。先关闭并重开终端，运行 `where.exe git`。若没有输出，检查 PATH 是否包含 `C:\Program Files\Git\cmd`；安装时 PATH 应选第二项。临时验证可执行 `$env:Path += ";C:\Program Files\Git\cmd"`，长期请用“系统属性 → 环境变量”修改，不要用 `setx` 拼接整个 PATH，容易截断。

02 装完新版本，`git --version` 还是旧的

先看 `where.exe git`（Windows）或 `which -a git`（macOS/Linux）的第一条路径。旧版本通常来自另一个安装渠道、IDE 自带 Git 或残留 PATH。关闭所有终端与 IDE 后重试；仍不对就调整 PATH 顺序，或卸载旧渠道。不要反复运行同一个安装包。

03 Git Bash 窗口一闪就关

从 PowerShell 运行 `& "C:\Program Files\Git\git-bash.exe"`，让错误留在窗口里；也可运行 `bash -lc "echo ok"` 检查 shell 是否正常。常见原因是用户目录权限、杀毒软件拦截、`~/.bashrc` 中有报错命令。临时用 `git-bash.exe --no-cd` 排除启动目录问题。

04 安装器提示无法写入 / 拒绝访问 / 安装到一半失败

确认以管理员身份运行、目标磁盘有空间、安装器未被安全软件隔离。升级时先关闭 Git Bash、VS Code、IDE 与构建任务。用静默参数加 `/LOG="%TEMP%\git-install.log"` 重试，日志会指出具体失败文件。仍失败时重启再装，不要直接删除 Program Files 下的半成品目录。

05 Git Bash 里能用，VS Code 终端里不能用

两者加载的 PATH 不同。VS Code 可能继承的是安装前的环境，或使用了不同的 shell。完全退出 VS Code 再打开；在 VS Code 终端运行 `where.exe git` 对比。若使用“Git Bash”配置文件，确认命令行指向 `C:\Program Files\Git\bin\bash.exe`，而不是旧的便携版。

9.2 提交、推送与换行符问题

06 **`fatal: not a git repository (or any of the parent directories)`**

当前目录不在任何 Git 仓库内，或 `.git` 被删除。运行 `pwd` 与 `ls -a`，确认看到 `.git`；用 `cd` 进入项目根目录。不要在子目录里再 `git init` 造一个嵌套仓库，那会让文件“消失”在另一个历史里。

07 **`nothing to commit, working tree clean`，但明明改了文件**

先 `git status` 看文件是否被忽略，再用 `git check-ignore -v <file>` 查规则。也可能改的是另一个副本，或文件在未保存的编辑器缓冲区里。若文件已被跟踪但改动被 `assume-unchanged` 隐藏，用 `git ls-files -v | grep '^h'` 检查。

08 **`warning: LF will be replaced by CRLF`**

这是换行符转换提示，不是失败。Windows 单平台可保持 `core.autocrlf=true`；跨平台团队改为 `input`，并在仓库加 `.gitattributes`（`* text=auto eol=lf`）。如果脚本在 Linux 报 `/bin/bash^M: bad interpreter`，说明文件带了 CRLF，按第 5.2 节统一处理，不要用 `dos2unix` 只修一台机器。

09 **`! [rejected] main -> main (non-fast-forward)`**

远程有你本地没有的提交。先 `git pull --rebase origin main`，解决冲突后再 `git push`。不要在共享分支上直接 `git push --force`；确需强制推送自己的功能分支时，用 `--force-with-lease`，并确认没有他人提交。

10 **`fatal: remote origin already exists`**

已经存在名为 `origin` 的远程。运行 `git remote -v` 查看地址；要改地址用 `git remote set-url origin <新地址>`，要改名字用 `git remote rename origin old-origin`。不要重复执行 `remote add`。

11 **`fatal: refusing to merge unrelated histories`**

两个仓库没有共同祖先，常见于“本地 init 后又 clone 了一个带 README 的远程仓库”。确认确实要把两段历史合并后，才使用 `git pull origin main --allow-unrelated-histories`。更简单的做法是重新 clone 远程，再把本地文件复制进去提交。

12 **提示 `HEAD detached at ...`，提交后不知道去哪了**

你处于“分离头指针”状态，提交不属于任何分支。若想保留当前提交，立即 `git switch -c rescue-work`；若想丢弃，`git switch main`。已经切换走也不一定丢，可用 `git reflog` 找回哈希，再建分支。

9.3 SSH、HTTPS 与网络问题

13 **`Permission denied (publickey)`**

按顺序检查：`ssh-add -l` 是否有密钥；`ssh -vT git@github.com` 是否提供了正确私钥；公钥是否已上传；`~/.ssh/config` 的 Host 是否匹配；权限是否为 `600/700`。多账号时加 `IdentitiesOnly yes`，避免 agent 里其他密钥抢先尝试。

14 **`Support for password authentication was removed`**

GitHub 不再接受账号密码。使用 SSH（第 6.2–6.4 节）或 Personal Access Token（第 6.5 节）。HTTPS 用户名填 GitHub 用户名，密码位置填令牌；配置 GCM 后通常会自动走浏览器登录。

15 **`SSL certificate problem: unable to get local issuer certificate`**

公司网络可能使用自签根证书，或系统 CA 不完整。让 IT 提供根证书 PEM，配置 `git config --global http.sslCAInfo "C:/certs/company-root-ca.pem"`。Windows Secure Channel 后端通常直接使用系统证书库。不要把 `http.sslVerify=false` 当成长期方案。

16 **`Could not resolve host` / 连接超时，但浏览器能打开 GitHub**

终端可能没有走系统代理。检查 `env | grep -i proxy`，配置 `http.proxy` / `https.proxy`；公司 VPN 需先连接。用 `curl -I https://github.com` 与 `git ls-remote` 分层测试。SSH 22 超时则改用 443（第 6.5 节）。

17 **`RPC failed; curl 92 HTTP/2 stream` / `early EOF`**

大仓库或弱网络下常见。先重试；仍失败可临时改用 HTTP/1.1：`git config --global http.version HTTP/1.1`，并适当增大 `http.postBuffer`。克隆大仓库可用 `--depth 1` 或 `--filter=blob:none`。若是单个大文件，考虑 Git LFS，而不是反复重试。

18 **`REMOTE HOST IDENTIFICATION HAS CHANGED`**

服务器主机密钥变了，可能是平台轮换，也可能是中间人。先向平台官方公告或管理员核实新指纹；确认后只删除 `known_hosts` 中对应主机的条目，再重新连接。不要直接清空整个 `known_hosts`，也不要关闭 `StrictHostKeyChecking`。

19 **`remote HEAD refers to nonexistent ref`**

远程仓库是空的，或默认分支还没建立。先在本地完成提交并 `git push -u origin main`；如果只是想克隆，确认远程地址指向的仓库不是空仓库，或向管理员确认默认分支名。

9.4 权限、长路径与其他问题

20 **`Filename too long`**

Windows 路径超过限制。执行 `git config --global core.longpaths true`，并在系统策略中启用 `LongPathsEnabled=1`；然后重新检出。若目录层级过深，也可以把仓库放到更短的根路径（如 `D:\code\`）。

21 **`detected dubious ownership in repository`**

仓库目录的所有者与当前用户不一致（常见于管理员打开、WSL/Windows 交叉访问、共享盘）。确认目录可信后，添加精确路径：`git config --global --add safe.directory D:/work/repo`。不要用 `safe.directory "*"` 全局放行，那会让恶意仓库绕过安全检查。

22 **Windows 上文件权限显示 `100644 -> 100755`，产生无意义改动**

Windows 文件系统不保留 Unix 可执行位，Git 可能把模式变化当成修改。在该仓库执行 `git config core.filemode false`，不要提交这些纯权限变更。若仓库确实需要可执行位（脚本、CI），应在 Linux/macOS 上设置并用 `git update-index --chmod=+x` 提交。

23 GCM 反复弹窗 / 每次都要重新登录

确认 `credential.helper` 是 `manager`；远程 URL 不要带过期用户名；清除 Windows 凭据管理器中对应的旧条目后重试。公司代理拦截 OAuth 时，GCM 可能无法完成浏览器回调，可改用 SSH 或按 IT 指引配置代理。不要为了省事把令牌写进 URL。

排错顺序：先证据，后改动

一条报错至少记录：完整命令、完整输出、`git --version`、`git --exec-path`、操作系统与 shell。然后只改一个变量并复测。把“我重装了三次”换成“我先确认了 PATH 第一条是什么”，通常会更快。

9.5 排错记录模板

如果问题需要同事、IT 或平台支持介入，复制下面这张表填写。它能避免“只发一张截图、没有版本、没有复现步骤”的低效沟通，也能帮你自己理清变量。

字段	填写内容
发生时间 / 频率	第一次出现的时间；每次必现，还是偶发？
完整命令	例如 `git push origin main`，不要只写“推送失败”。
完整输出	从第一行错误到最后的提示；隐去令牌、邮箱等隐私。
环境	操作系统与版本、Git 版本、`git --exec-path`、shell（PowerShell / Git Bash / zsh）。
已尝试	改过哪些配置、是否重装、是否换网络；每次只改一项。
最近变化	是否刚升级 Git、换电脑、改代理、换 SSH 密钥、加入新组织。

表 9-1 排错记录模板。能稳定复现的命令与输出，比“我觉得是 Git 坏了”有价值得多。

分享日志前先脱敏

`GIT_CURL_VERBOSE` 与凭据助手日志可能包含远程 URL、用户名、代理地址，甚至令牌片段。粘贴到 Issue 或聊天群前，先搜索并替换 `ghp_`、`glpat-`、`@` 后面的敏感信息。

MAINTENANCE / SECURITY

10

维护与安全

Git 装好之后不会自动保持“正确”。版本会更新，密钥会过期，令牌会泄露，旧电脑会被转手。把升级、备份、轮换与卸载当成日常维护的一部分，比出问题后抢救更省力。

预计 15 分钟 | 长期适用 | 含迁移清单

10.1 升级策略与版本管理

Git 的发版节奏较快，次版本通常带来功能与性能改进，修订号修缺陷与安全问题。个人电脑可以跟随稳定版；团队与 CI 应固定一个可复现的版本，并在升级前跑一遍构建与测试。

平台	升级方式	升级后验证
Windows	`git update-git-for-windows` 或 `winget upgrade Git.Git`	`git --version` + `where.exe git`
macOS（Homebrew）	`brew update && brew upgrade git`	`hash -r` + `which -a git`
macOS（官方 pkg）	下载新版 `.pkg` 覆盖安装	`git --version`
Ubuntu / Debian	`sudo apt update && sudo apt upgrade git`	`apt policy git` + `git --version`
Fedora / RHEL	`sudo dnf upgrade git`	`rpm -q git` + `git --version`
Arch	`sudo pacman -Syu git`	`pacman -Q git`

表 10-1 各平台升级方式。升级前关闭终端与 IDE，避免文件占用。

升级前的五分钟检查

- ☐ **读发布说明。**至少看“Breaking changes / Behavior changes”，不要只看版本号。
- ☐ **备份配置。**`~/.gitconfig`、`~/.ssh` 与项目清单。
- ☐ **清理工作区。**提交或 stash 未完成的改动，避免升级中断。
- ☐ **关闭占用。**IDE、终端、Git GUI、构建任务全部退出。
- ☐ **准备回滚。**保留旧安装包或包管理器缓存，知道怎么装回旧版。
- ☐ **先测一台。**团队环境先在测试机或 CI 跑一遍构建与钩子。

> **“最新”不等于“最适合生产”**
>
> 生产服务器优先跟随发行版安全更新，不必追每个次版本。若项目需要固定版本，把 `git --version` 写进 CI 日志与开发环境文档；升级时按变更单执行，而不是让某台机器悄悄变成另一个版本。

10.2 配置、密钥与令牌安全

Git 的配置文件通常不含密码，但它可能包含公司邮箱、内部域名、代理地址与 include 路径。真正敏感的是 SSH 私钥、访问令牌、凭据管理器中的缓存，以及误提交的 `.env`。

对象	风险	建议
SSH 私钥	被复制即可冒充你访问所有已授权仓库	设置口令；每台设备单独一把；离职/丢失设备立即从平台删除公钥。
PAT / 访问令牌	泄露后可读写仓库，甚至触发 CI	最小权限、短期有效、按用途分开；不要把令牌写进 URL、脚本或仓库。
凭据管理器	共享电脑上他人可直接使用	使用系统账户隔离；离开电脑锁屏；共用机器用临时令牌并及时清除。
`~/.gitconfig`	泄露内部域名、代理、邮箱	备份到加密位置；不要在公开 Issue 中粘贴完整配置。
仓库中的 `.env`	数据库密码、API Key 进入历史	首次提交前加入 `.gitignore`；泄露后立即轮换密钥。

表 10-2 Git 相关敏感对象。发现泄露时，先轮换，再清理历史；顺序不能反。

密钥轮换

1. 生成新密钥（第 6.2 节），设置新口令。
2. 把新公钥上传到平台，保留旧公钥一段时间用于过渡。
3. 在本机测试 `ssh -T` 与一次 `git fetch`。
4. 确认所有自动化与仓库都切换后，从平台删除旧公钥。
5. 删除本机旧私钥，清理 `~/.ssh/config` 与 agent 中的旧条目。

签名提交（可选）

如果团队要求证明“提交确实来自你”，可以启用提交签名。Git 支持 GPG 与 SSH 两种格式；GitHub / GitLab 都能显示“Verified”。最小配置示例（使用已有 SSH 密钥）：

```
$ git config --global gpg.format ssh
$ git config --global user.signingkey ~/.ssh/id_ed25519_github.pub
$ git config --global commit.gpgsign true

# 测试签名提交
$ git commit -S -m "test: signed commit"
```

还要把同一把公钥作为“Signing Key”上传到平台，否则平台只能看到签名，无法确认归属。签名会增加一点操作复杂度；个人项目不必强求，团队有合规要求时再启用。

不要把“清理历史”当成泄露后的第一步

密钥已经推送到远程，说明它应当被视为已泄露。先吊销/轮换，再评估是否用 `git filter-repo` 清理历史。清理历史会改写所有提交哈希，影响协作者；必须提前通知并协调。仅删除最新文件不会让旧提交中的密钥消失。

10.3 干净卸载与迁移

卸载 Git 与删除项目是两件事。卸载程序通常只移除程序文件与系统 PATH，不会删除你的仓库、全局配置与 SSH 密钥；迁移时正好可以利用这一点。

平台	卸载方式	保留 / 检查
Windows	设置 → 应用 → Git → 卸载	保留 `%USERPROFILE%\.gitconfig`、`.ssh`；检查 PATH 残留。
macOS（Homebrew）	`brew uninstall git`	保留 `~/.gitconfig`、`~/.ssh`；系统 `/usr/bin/git` 不动。
macOS（官方 pkg）	运行 `/usr/local/git/uninstall.sh`（若存在）	不同版本脚本位置可能不同，先确认再执行。
Debian / Ubuntu	`sudo apt remove git`	`apt autoremove` 清理依赖；配置与密钥保留。
源码安装	在源码目录 `sudo make prefix=/usr/local uninstall`	源码目录删除后无法卸载，只能按清单手动清理。

表 10-3 卸载方式。卸载前确认没有正在进行的 Git 操作，仓库已推送或备份。

迁移到新电脑

- ☐ **仓库。**优先重新 clone；本地未推送的分支先 push 或打包。
- ☐ **SSH 密钥。**安全传输私钥，权限设为 `600`；或在新机重新生成并上传公钥。
- ☐ **工具链。**确认 Git、SSH、编辑器、GPG/签名工具都已安装。
- ☐ **全局配置。**复制 `.gitconfig` 与 include 文件，检查路径是否存在。
- ☐ **凭据。**不要复制旧机器的凭据缓存；在新机重新登录 GCM / 钥匙串。
- ☐ **验收。**在新机跑第 8 章十项检查，再做一次真实 clone / push。

迁移后如果提交 Authors 显示的是旧名字，检查 `git var GIT_AUTHOR_IDENT` 与仓库 local 配置；如果推送提示权限错误，先确认新机 SSH 指纹与公钥，再检查平台上的设备授权。不要直接把旧电脑的 `.ssh` 整个网盘同步到新电脑——私钥应通过加密介质或密码管理器传输。

END OF MAIN TEXT

到这里，安装、配置、认证、首个仓库、验收与维护已经形成闭环。接下来的三个附录可以直接打印或复制：命令速查、配置模板、官方资源与术语表。

APPENDIX A / CHEAT SHEET

A

命令速查表

按“配置 → 提交 → 分支 → 远程 → 撤销”分组。方括号表示可选参数，尖括号表示必须替换的占位符。建议打印成 A4 放在手边，而不是收藏一份永远不看的长文。

可打印 32 条常用命令 Git 2.40+

A.1 初始化与配置

命令	作用
`git init`	在当前目录创建仓库。
`git clone <url> [dir]`	克隆远程仓库到指定目录。
`git config --global user.name "姓名"`	设置全局提交姓名。
`git config --global user.email "邮箱"`	设置全局提交邮箱。
`git config --list --show-origin --show-scope`	查看生效配置及来源。
`git config --global --edit`	用编辑器打开全局配置。

A.2 日常提交

命令	作用
`git status -sb`	简洁查看分支与文件状态。
`git add <file>`	把指定文件放入暂存区。
`git add -A`	暂存所有新增、修改与删除。
`git diff`	查看工作区与暂存区的差异。
`git diff --staged`	查看暂存区与上次提交的差异。
`git commit -m "type: message"`	提交暂存内容。
`git commit --amend`	修改上一条未推送的提交。
`git log --oneline --graph --decorate --all`	图形化查看历史。

A.3 分支与合并

命令	作用
`git branch`	列出本地分支。
`git switch -c <branch>`	创建并切换到新分支。
`git switch <branch>`	切换分支。
`git merge <branch>`	把指定分支合并到当前分支。
`git rebase <branch>`	把当前分支变基到指定分支。
`git branch -d <branch>`	删除已合并的本地分支。
`git push -u origin <branch>`	推送分支并建立跟踪。
`git fetch --prune`	拉取远程更新并清理失效引用。

A.4 远程与撤销

命令	作用
`git remote -v`	查看远程地址。
`git remote add origin <url>`	添加名为 origin 的远程。
`git remote set-url origin <url>`	修改 origin 地址。
`git pull --rebase`	拉取并变基，保持线性历史。
`git push`	推送当前分支。
`git restore <file>`	丢弃工作区修改（不可恢复）。
`git restore --staged <file>`	把文件移出暂存区。
`git revert <hash>`	生成反向提交，撤销已推送提交。
`git reset --soft HEAD~1`	撤销本地提交，保留改动。
`git reflog`	查看 HEAD 移动记录，找回误操作。

表 A-1 常用命令速查。带 `--hard`、`--force` 的命令不在表中鼓励使用，执行前务必确认影响范围。

APPENDIX B / CONFIG TEMPLATE

B

推荐配置模板

下面是一份偏“稳妥”的全局配置起点。先复制，再按平台修改 `core.autocrlf`、`credential.helper` 与编辑器路径；不要不加判断地整套覆盖现有配置。

可直接复制 | Windows / macOS / Linux | 含注释

模板适用于 Git 2.40+，其中 `push.autoSetupRemote` 需要 Git 2.37+。保存为 `~/.gitconfig`（Windows 为 `%USERPROFILE%\.gitconfig`），或把需要的段落合并到现有文件。

B.1 基础配置

~/.gitconfig

```
# 身份：改成你自己的姓名与邮箱
[user]
    name = Zhang San
    email = zhangsan@example.com
    # useConfigOnly = true

# 新仓库默认分支
[init]
    defaultBranch = main

[core]
    # Windows 单平台用 true；跨平台用 input
    autocrlf = true
    # 中文文件名不转义
    quotepath = false
    # Windows 长路径；macOS/Linux 可删除
    longpaths = true

[color]
    ui = auto

# Windows 用 manager；macOS 用 osxkeychain
[credential]
    helper = manager
```

B.2 协作、性能与别名

以下段落接在同一文件末尾，不需要另建文件。

~/.gitconfig（续）

```
[push]
    default = simple
    autoSetupRemote = true

[fetch]
    prune = true

[rerere]
    enabled = true

[alias]
    st = status -sb
    lg = log --oneline --graph --decorate --all
    last = log -1 --stat
```

按平台替换的配置

平台	core.autocrlf	credential.helper
Windows 单平台	`true`	`manager`
Windows 跨平台	`input`	`manager`
macOS	`input`（或 `false`）	`osxkeychain`
Linux 桌面	`input`（或 `false`）	`libsecret`
Linux 服务器	`input`	`cache --timeout=3600`（临时）

表 B-1 平台差异只在这两三项；其余配置可以共用。

可选：公司与个人身份自动切换

~/.gitconfig（追加）

```
[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work

# ~/.gitconfig-work
[user]
    name = Zhang San
    email = zhangsan@company.com
```

> **模板不是“越全越好”**
>
> 不要为了“看起来专业”加入大量自己不理解的颜色、分页器、钩子与代理配置。每增加一项，就多一个出问题时需要排查的变量。先保证身份、换行符、凭据与默认分支正确，再按实际痛点增加。

APPENDIX C / REFERENCES

C

官方资源与术语表

安装与配置会随版本变化，遇到不确定的选项，优先查官方文档与发行说明。下面列出本指南涉及的主要资源，以及阅读 Git 文档时最常见的术语。

链接可点击 | 术语 18 条 | 版本基线 2.56.0

C.1 官方资源

Git 官网与文档
https://git-scm.com/doc
命令参考、Pro Git 在线版、发行说明入口。

Pro Git（中文）
https://git-scm.com/book/zh/v2
从入门到内部的系统教材，适合补概念。

git-config 手册
https://git-scm.com/docs/git-config
查配置项的含义、默认值与作用域。

Git for Windows
https://gitforwindows.org/
Windows 安装器、Git Bash、发行说明。

Homebrew
https://brew.sh/
macOS / Linux 包管理器；Git 安装与升级。

GitHub SSH 文档
https://docs.github.com/zh/authentication/connecting-to-github-with-ssh
生成密钥、添加公钥、测试连接、SSH over 443。

GitLab SSH 文档
https://docs.gitlab.com/user/ssh/
GitLab 的密钥类型、过期时间与故障排查。

Gitee SSH 帮助
https://gitee.com/help/articles/4181
Gitee 公钥设置与常见问题。

Conventional Commits
https://www.conventionalcommits.org/zh-hans/
提交信息前缀约定，团队协作可选。

C.2 术语表

术语	解释
仓库 / Repository	包含项目文件与 `.git` 历史的目录；本地仓库与远程仓库是两份副本。
工作区 / Working Tree	你当前能看到、能编辑的文件集合。
暂存区 / Index	下一次提交要包含的快照；`git add` 把改动放进去。
提交 / Commit	一次不可变的快照，包含作者、时间、父提交与提交信息。
分支 / Branch	指向某个提交的可移动指针；切换分支会改变工作区内容。
HEAD	当前检出的提交或分支的引用；分离头指针时直接指向提交。
远程 / Remote	另一个仓库的地址与引用；默认名字常为 `origin`。
跟踪分支 / Upstream	本地分支关联的远程分支，决定 `pull` / `push` 的默认目标。
合并 / Merge	把两条历史线合并，可能产生合并提交。
变基 / Rebase	把一串提交重新应用到新基底上，历史更线性，但会改写哈希。
冲突 / Conflict	同一区域有无法自动合并的改动，需要人工选择最终内容。
标签 / Tag	给某个提交起的固定名字，常用于版本发布。
Stash	临时保存未提交改动，以便切换分支或拉取更新。
PAT	Personal Access Token，HTTPS 认证用的令牌，可限制权限与有效期。
GCM	Git Credential Manager，Windows 上的凭据助手，支持 OAuth 与 2FA。
LFS	Large File Storage，把大文件替换为指针，内容单独存储。
Safe Directory	Git 对“目录所有者与当前用户不一致”的安全检查；只放行可信路径。
Renormalize	按 `.gitattributes` 重新规范化已跟踪文件的换行符。

表 C-1 阅读 Git 文档时的高频术语。遇到陌生词，先回到“工作区 / 暂存区 / 提交”三层模型定位。

AFTER INSTALLATION

装好只是起点，用对才是目的。

版本可追溯、配置可复现、认证可撤销。把这三件事做好，Git 就不会成为项目里的黑盒。

01 知道自己在用哪一套 Git

02 把配置写进可复制的文件

03 让密钥与令牌随时可轮换

Git 安装与配置完全指南

v1.0 · 2026 年 10 月 · 基于 Git 2.56.0

git.